01_week1_poly_interactions

June 28, 2026

Each week, you will apply the concepts of that week to your Integrated Capstone Project's dataset. In preparation for Milestone One, create a Jupyter Notebook (similar to in Module B, semester two) that illustrates these lessons. There are no specific questions to answer in your Jupyter Notebook files in this course; your general goal is to analyze your data, using the methods you have learned about in this course and in this program, and draw interesting conclusions.

For Week 1, include concepts such as linear regression with polynomial terms, interaction terms, multicollinearity, variance inflation factor and regression, and categorical and continuous features. Complete your Jupyter Notebook homework by 11:59 pm ET on Sunday.

In Week 7, you will compile your findings from your Jupyter Notebook homework into your Milestone One assignment for grading. For full instructions and the rubric for Milestone One, refer to the following link.

sets the proper root directory so helper load function works

[1]:
```
from pathlib import Path
import sys

PROJECT_ROOT = Path.cwd().resolve().parents[1]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.append(str(PROJECT_ROOT))

print(PROJECT_ROOT)
```

```
/Users/jamieconner/projects/bu/dx799_01/mod_c_capstone
```

Load the data using my helper functions to both load and split wisconsin data

[2]:
```
from src.load_data import load_wisconsin, load_brazil, split_X_y


df_wisc, target = load_wisconsin()
X, y = split_X_y(df_wisc, target)

X.describe()
```

[2]:

	radius1	texture1	perimeter1	area1	smoothness1	\
count	569.000000	569.000000	569.000000	569.000000	569.000000	
mean	14.127292	19.289649	91.969033	654.889104	0.096360	
std	3.524049	4.301036	24.298981	351.914129	0.014064	

```
min      6.981000    9.710000   43.790000   143.500000     0.052630
25%     11.700000   16.170000   75.170000   420.300000     0.086370
50%     13.370000   18.840000   86.240000   551.100000     0.095870
75%     15.780000   21.800000  104.100000   782.700000     0.105300
max     28.110000   39.280000  188.500000  2501.000000     0.163400

       compactness1  concavity1  concave_points1   symmetry1  \
count    569.000000  569.000000       569.000000  569.000000
mean       0.104341    0.088799         0.048919    0.181162
std        0.052813    0.079720         0.038803    0.027414
min        0.019380    0.000000         0.000000    0.106000
25%        0.064920    0.029560         0.020310    0.161900
50%        0.092630    0.061540         0.033500    0.179200
75%        0.130400    0.130700         0.074000    0.195700
max        0.345400    0.426800         0.201200    0.304000

       fractal_dimension1  …     radius3    texture3  perimeter3  \
count          569.000000  …  569.000000  569.000000  569.000000
mean             0.062798  …   16.269190   25.677223  107.261213
std              0.007060  …    4.833242    6.146258   33.602542
min              0.049960  …    7.930000   12.020000   50.410000
25%              0.057700  …   13.010000   21.080000   84.110000
50%              0.061540  …   14.970000   25.410000   97.660000
75%              0.066120  …   18.790000   29.720000  125.400000
max              0.097440  …   36.040000   49.540000  251.200000

             area3  smoothness3  compactness3  concavity3  concave_points3  \
count   569.000000   569.000000    569.000000  569.000000       569.000000
mean    880.583128     0.132369      0.254265    0.272188         0.114606
std     569.356993     0.022832      0.157336    0.208624         0.065732
min     185.200000     0.071170      0.027290    0.000000         0.000000
25%     515.300000     0.116600      0.147200    0.114500         0.064930
50%     686.500000     0.131300      0.211900    0.226700         0.099930
75%    1084.000000     0.146000      0.339100    0.382900         0.161400
max    4254.000000     0.222600      1.058000    1.252000         0.291000

        symmetry3  fractal_dimension3
count  569.000000          569.000000
mean     0.290076            0.083946
std      0.061867            0.018061
min      0.156500            0.055040
25%      0.250400            0.071460
50%      0.282200            0.080040
75%      0.317900            0.092080
max      0.663800            0.207500

[8 rows x 30 columns]
```

train test split data for baseline regression model

[3]:
```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
import numpy as np


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.2,␣
 ↪random_state=42, stratify=y)


base_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))
])

cv_scores = cross_val_score(
    base_pipeline,
    X_train,
    y_train,
    cv=5,
    scoring= "roc_auc"
)


baseline_mean_train_cv = np.mean(cv_scores)

print(f"baseline model training cv: {baseline_mean_train_cv}")
```

```
baseline model training cv: 0.9951496388028895
```

for simplicity and noise reduction in polynomial intereaction i'm going to subset a few features from the full data set - choosing features that represent size, shape, texture, boundary formatioon

[4]:
```
subset_feat= ["radius1", "symmetry1", "compactness1", "texture1",␣
 ↪"concave_points1"]


X_train_subset = X_train[subset_feat]
X_tests_subset = X_test[subset_feat]

b_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))
])
```

```
cv_scores = cross_val_score(
    base_pipeline,
    X_train_subset,
    y_train,
    cv=5,
    scoring= "roc_auc"
)


subset_mean_train_cv = np.mean(cv_scores)

print(f"baseline subset model training cv: {subset_mean_train_cv}")
```

```
baseline subset model training cv: 0.9844169246646027
```

add in polyterms

[5]:
```
from sklearn.preprocessing import PolynomialFeatures


poly_pipeline = Pipeline([
    ("polynomial", PolynomialFeatures((2,3))),
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))
])

cv_scores = cross_val_score(
    poly_pipeline,
    X_train_subset,
    y_train,
    cv=5,
    scoring= "roc_auc"
)


subset_poly_mean_train_cv = np.mean(cv_scores)

print(f"baseline subset poly model training cv: {subset_poly_mean_train_cv}")
```

```
baseline subset poly model training cv: 0.9882352941176471
```

[6]:
```
print(f"baseline model training cv: {baseline_mean_train_cv}")

print(f"baseline subset model training cv: {subset_mean_train_cv}")


print(f"baseline subset poly model training cv: {subset_poly_mean_train_cv}")
```

```
baseline model training cv: 0.9951496388028895
```

```
baseline subset model training cv: 0.9844169246646027
baseline subset poly model training cv: 0.9882352941176471
```

I"m going to test all three models on the test set. I know in general we supposed to pick on final model to use on test, but i'm not going to change my models after the fact, just one final comparison to see how the models generalize.

[7]:
```python
from sklearn.metrics import accuracy_score, roc_auc_score

# full-feature baseline
base_pipeline.fit(X_train, y_train)

y_pred = base_pipeline.predict(X_test)
y_prob = base_pipeline.predict_proba(X_test)[:, 1]

base_test_accuracy = accuracy_score(y_test, y_pred)
base_test_roc_auc = roc_auc_score(y_test, y_prob)

print(f"Full baseline test accuracy: {base_test_accuracy:.4f}")
print(f"Full baseline test ROC-AUC: {base_test_roc_auc:.4f}")


# subset baseline
base_pipeline.fit(X_train_subset, y_train)

y_pred_subset = base_pipeline.predict(X_tests_subset)
y_prob_subset = base_pipeline.predict_proba(X_tests_subset)[:, 1]

subset_test_accuracy = accuracy_score(y_test, y_pred_subset)
subset_test_roc_auc = roc_auc_score(y_test, y_prob_subset)

print(f"Subset baseline test accuracy: {subset_test_accuracy:.4f}")
print(f"Subset baseline test ROC-AUC: {subset_test_roc_auc:.4f}")


# subset polynomial model
poly_pipeline.fit(X_train_subset, y_train)

y_pred_poly = poly_pipeline.predict(X_tests_subset)
y_prob_poly = poly_pipeline.predict_proba(X_tests_subset)[:, 1]

poly_test_accuracy = accuracy_score(y_test, y_pred_poly)
poly_test_roc_auc = roc_auc_score(y_test, y_prob_poly)

print(f"Subset polynomial test accuracy: {poly_test_accuracy:.4f}")
print(f"Subset polynomial test ROC-AUC: {poly_test_roc_auc:.4f}")
```

```
Full baseline test accuracy: 0.9649
Full baseline test ROC-AUC: 0.9960
```

```
Subset baseline test accuracy: 0.9298
Subset baseline test ROC-AUC: 0.9858
Subset polynomial test accuracy: 0.9474
Subset polynomial test ROC-AUC: 0.9888
```

To satisfy Milestone 1 breadth, I also applied the same Week 1 comparison to the Brazil dataset. Because the Brazil file is much larger, I used an 8,000-row stratified sample to keep the notebook runnable. I kept the same three-model structure: a full-feature baseline logistic model, a reduced five-feature baseline, and a polynomial expansion of that reduced set.

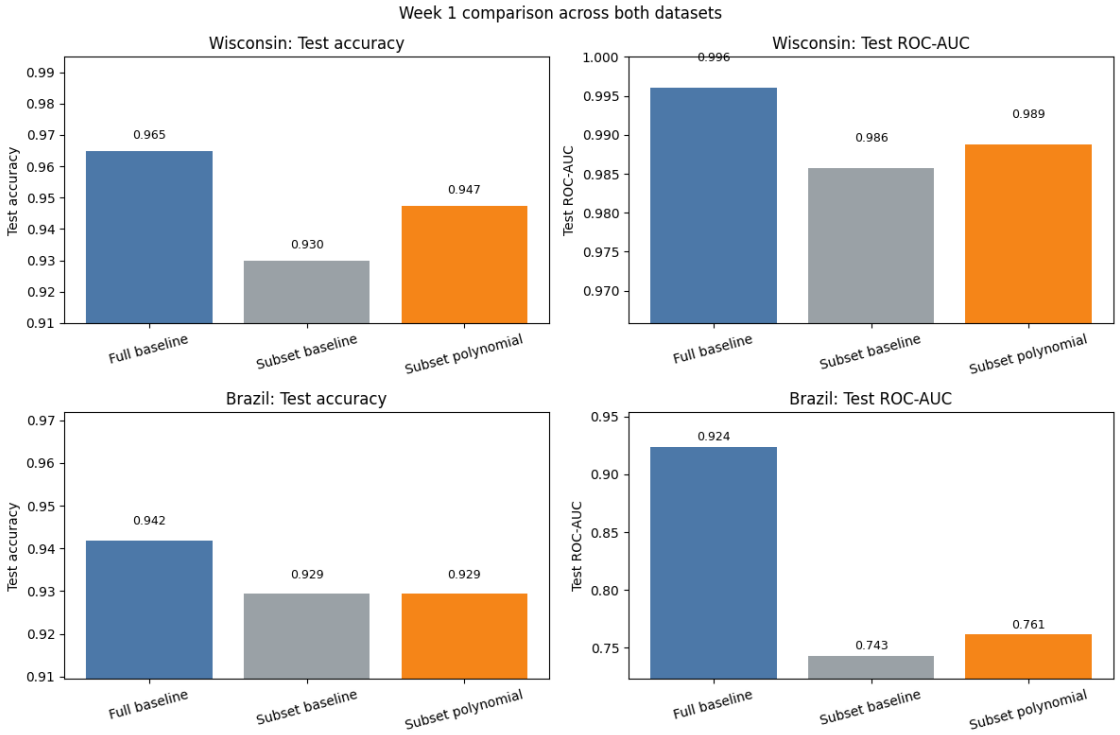
[8]:
```
df_brazil, target_brazil = load_brazil(sample_size=8000, random_state=42)
X_brazil, y_brazil = split_X_y(df_brazil, target_brazil)

X_train_brazil, X_test_brazil, y_train_brazil, y_test_brazil = train_test_split(
    X_brazil, y_brazil, test_size=0.2, random_state=42, stratify=y_brazil
)

brazil_subset_feat = [
    'age',
    'morphology_code',
    'gender_female',
    'diagnostic_method_primary_tumor_histology',
    'extension_metastasis',
]

X_train_brazil_subset = X_train_brazil[brazil_subset_feat]
X_test_brazil_subset = X_test_brazil[brazil_subset_feat]

brazil_cv_scores = cross_val_score(
    base_pipeline,
    X_train_brazil,
    y_train_brazil,
    cv=5,
    scoring="roc_auc",
)
brazil_baseline_mean_train_cv = np.mean(brazil_cv_scores)

brazil_cv_scores = cross_val_score(
    base_pipeline,
    X_train_brazil_subset,
    y_train_brazil,
    cv=5,
    scoring="roc_auc",
)
brazil_subset_mean_train_cv = np.mean(brazil_cv_scores)

brazil_cv_scores = cross_val_score(
    poly_pipeline,
```

```
    X_train_brazil_subset,
    y_train_brazil,
    cv=5,
    scoring="roc_auc",
)
brazil_subset_poly_mean_train_cv = np.mean(brazil_cv_scores)

base_pipeline.fit(X_train_brazil, y_train_brazil)
y_pred_brazil = base_pipeline.predict(X_test_brazil)
y_prob_brazil = base_pipeline.predict_proba(X_test_brazil)[:, 1]
brazil_base_test_accuracy = accuracy_score(y_test_brazil, y_pred_brazil)
brazil_base_test_roc_auc = roc_auc_score(y_test_brazil, y_prob_brazil)

base_pipeline.fit(X_train_brazil_subset, y_train_brazil)
y_pred_brazil_subset = base_pipeline.predict(X_test_brazil_subset)
y_prob_brazil_subset = base_pipeline.predict_proba(X_test_brazil_subset)[:, 1]
brazil_subset_test_accuracy = accuracy_score(y_test_brazil,␣
 ↪y_pred_brazil_subset)
brazil_subset_test_roc_auc = roc_auc_score(y_test_brazil, y_prob_brazil_subset)

poly_pipeline.fit(X_train_brazil_subset, y_train_brazil)
y_pred_brazil_poly = poly_pipeline.predict(X_test_brazil_subset)
y_prob_brazil_poly = poly_pipeline.predict_proba(X_test_brazil_subset)[:, 1]
brazil_poly_test_accuracy = accuracy_score(y_test_brazil, y_pred_brazil_poly)
brazil_poly_test_roc_auc = roc_auc_score(y_test_brazil, y_prob_brazil_poly)

print(f"Brazil full baseline training CV ROC-AUC:␣
 ↪{brazil_baseline_mean_train_cv:.4f}")
print(f"Brazil subset baseline training CV ROC-AUC:␣
 ↪{brazil_subset_mean_train_cv:.4f}")
print(f"Brazil subset polynomial training CV ROC-AUC:␣
 ↪{brazil_subset_poly_mean_train_cv:.4f}")
print()
print(f"Brazil full baseline test accuracy: {brazil_base_test_accuracy:.4f}")
print(f"Brazil full baseline test ROC-AUC: {brazil_base_test_roc_auc:.4f}")
print(f"Brazil subset baseline test accuracy: {brazil_subset_test_accuracy:.
 ↪4f}")
print(f"Brazil subset baseline test ROC-AUC: {brazil_subset_test_roc_auc:.4f}")
print(f"Brazil subset polynomial test accuracy: {brazil_poly_test_accuracy:.
 ↪4f}")
print(f"Brazil subset polynomial test ROC-AUC: {brazil_poly_test_roc_auc:.4f}")
```

```
Brazil full baseline training CV ROC-AUC: 0.9277
Brazil subset baseline training CV ROC-AUC: 0.7870
Brazil subset polynomial training CV ROC-AUC: 0.7869

Brazil full baseline test accuracy: 0.9419
```

```
Brazil full baseline test ROC-AUC: 0.9237
Brazil subset baseline test accuracy: 0.9294
Brazil subset baseline test ROC-AUC: 0.7428
Brazil subset polynomial test accuracy: 0.9294
Brazil subset polynomial test ROC-AUC: 0.7615
```

[9]:
```python
import matplotlib.pyplot as plt

model_labels = ['Full baseline', 'Subset baseline', 'Subset polynomial']
colors = ['#4C78A8', '#9AA1A6', '#F58518']

dataset_metrics = {
    'Wisconsin': {
        'Test accuracy': [base_test_accuracy, subset_test_accuracy,␣
 ↪poly_test_accuracy],
        'Test ROC-AUC': [base_test_roc_auc, subset_test_roc_auc,␣
 ↪poly_test_roc_auc],
    },
    'Brazil': {
        'Test accuracy': [brazil_base_test_accuracy,␣
 ↪brazil_subset_test_accuracy, brazil_poly_test_accuracy],
        'Test ROC-AUC': [brazil_base_test_roc_auc, brazil_subset_test_roc_auc,␣
 ↪brazil_poly_test_roc_auc],
    },
}

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for row_idx, (dataset_name, metric_map) in enumerate(dataset_metrics.items()):
    for col_idx, (metric_name, values) in enumerate(metric_map.items()):
        ax = axes[row_idx, col_idx]
        bars = ax.bar(model_labels, values, color=colors)
        y_min = min(values) - 0.02
        y_max = min(1.0, max(values) + 0.03)
        ax.set_ylim(max(0, y_min), y_max)
        ax.set_title(f"{dataset_name}: {metric_name}")
        ax.set_ylabel(metric_name)
        ax.tick_params(axis="x", rotation=15)
        for bar, value in zip(bars, values):
            ax.text(bar.get_x() + bar.get_width() / 2, value + 0.003, f"{value:.
 ↪3f}", ha="center", va="bottom", fontsize=9)

fig.suptitle('Week 1 comparison across both datasets', y=0.98)
fig.tight_layout()
plt.show()
```



For Week 1, I compared three regression-based approaches on both cancer datasets: a full-feature baseline logistic model, a reduced five-feature baseline, and a polynomial expansion of that reduced set. On the Wisconsin dataset, the full-feature baseline performed best on the held-out test set, with accuracy of 0.9649 and ROC-AUC of 0.9960. Reducing the model to five features lowered performance, while adding polynomial and interaction terms improved that reduced model somewhat without beating the full original-feature baseline. This fits the broader Wisconsin pattern seen throughout the project: the dataset is already clean and highly separable, so extra nonlinear complexity only adds limited value.

The Brazil results told a similar but noisier story. The full-feature baseline also performed best there, with test accuracy of 0.9419 and ROC-AUC of 0.9237. Using only a five-feature subset dropped ROC-AUC substantially, and adding polynomial terms improved that reduced model from about 0.7428 to 0.7615 ROC-AUC, but still left it well below the full-feature baseline. That suggests interaction terms can recover some signal in a smaller Brazil model, but the larger and more heterogeneous dataset benefits much more from keeping the richer full feature set. Across both datasets, polynomial expansion helped the reduced model but did not justify replacing the simpler full-feature baseline.

02_week2_regularization

June 28, 2026

Each week, you will apply the concepts of that week to your Integrated Capstone Project's dataset. In preparation for Milestone One, create a Jupyter Notebook (similar to in Module B, semester two) that illustrates these lessons. There are no specific questions to answer in your Jupyter Notebook files in this course; your general goal is to analyze your data, using the methods you have learned about in this course and in this program, and draw interesting conclusions.

For Week 2, include concepts such as linear regression with lasso, ridge, and elastic net regression. This homework will be submitted for peer review and feedback in Week 3 in the assignment titled 3.4 Peer Review: Week 2 Jupyter Notebook. Complete your Jupyter Notebook homework by 11:59 pm ET on Sunday.

In Week 7, you will compile your findings from your Jupyter Notebook homework into your Milestone One assignment for grading. For full instructions and the rubric for Milestone One, refer to the following link.

[1]:
```
from pathlib import Path
import sys

PROJECT_ROOT = Path.cwd().resolve().parents[1]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.append(str(PROJECT_ROOT))

print(PROJECT_ROOT)
```

```
/Users/jamieconner/projects/bu/dx799_01/mod_c_capstone
```

[2]:
```
from src.load_data import load_wisconsin, load_brazil, split_X_y

df_brazil, target = load_brazil(sample_size= 50_000)
X_brazil, y_brazil = split_X_y(df_brazil, target)
```

[3]:
```
X_brazil.info()
y_brazil
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 50000 entries, 700187 to 754772
Data columns (total 39 columns):
 #   Column                                    Non-Null Count  Dtype
---  ------                                    --------------  -----
 0   age                                       50000 non-null  float64
```

```
 1   profession_code                          50000 non-null  float64
 2   morphology_code                          50000 non-null  int64
 3   gender_female                            50000 non-null  int64
 4   gender_unknown                           50000 non-null  int64
 5   gender_male                              50000 non-null  int64
 6   race_color_yellow                        50000 non-null  int64
 7   race_color_white                         50000 non-null  int64
 8   race_color_indigenous                    50000 non-null  int64
 9   race_color_brown                         50000 non-null  int64
 10  race_color_black                         50000 non-null  int64
 11  education_elementary_1_to_4              50000 non-null  int64
 12  education_elementary_5_to_8              50000 non-null  int64
 13  education_secondary                      50000 non-null  int64
 14  education_no_schooling                   50000 non-null  int64
 15  education_college_complete               50000 non-null  int64
 16  education_college_incomplete             50000 non-null  int64
 17  marital_status_married                   50000 non-null  int64
 18  marital_status_legally_separated         50000 non-null  int64
 19  marital_status_single                    50000 non-null  int64
 20  marital_status_common_law_union          50000 non-null  int64
 21  marital_status_widowed                   50000 non-null  int64
 22  rare_case_false                          50000 non-null  int64
 23  rare_case_true                           50000 non-null  int64
 24  diagnostic_method_cytology               50000 non-null  int64
 25  diagnostic_method_clinical               50000 non-null  int64
 26  diagnostic_method_metastasis_histology   50000 non-null  int64
 27  diagnostic_method_primary_tumor_histology 50000 non-null  int64
 28  diagnostic_method_tumor_markers          50000 non-null  int64
 29  diagnostic_method_research               50000 non-null  int64
 30  diagnostic_method_sdo                    50000 non-null  int64
 31  extension_in_situ                        50000 non-null  int64
 32  extension_localized                      50000 non-null  int64
 33  extension_metastasis                     50000 non-null  int64
 34  extension_not_applicable                 50000 non-null  int64
 35  laterality_bilateral                     50000 non-null  int64
 36  laterality_right                         50000 non-null  int64
 37  laterality_left                          50000 non-null  int64
 38  laterality_not_applicable                50000 non-null  int64
dtypes: float64(2), int64(37)
memory usage: 15.3 MB
```

[3]:
```
700187     0
1473092    0
1119056    0
1614127    0
1714606    0
          ..
```

```
603780    1
278573    0
768371    0
558743    0
754772    0
Name: deceased, Length: 50000, dtype: int64
```

[ ]:

[4]:
```
from sklearn.model_selection import train_test_split, cross_val_score,␣
 ↪GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
import pandas as pd


X_train, X_test, y_train, y_test = train_test_split(
    X_brazil,
    y_brazil,
    random_state=42,
    test_size=.2,
    stratify = y_brazil
    )

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=3000, solver="saga",␣
 ↪class_weight="balanced"))
])


grid_params = [
    {"model__penalty":["l2"], "model__C": [0.01, 0.1, 1, 10]},
    {"model__penalty":["l1"], "model__C": [0.01, 0.1, 1, 10]},

    {
      "model__penalty":["elasticnet"],
      "model__C": [0.01, 0.1, 1, 10],
      "model__l1_ratio": [0.2, 0.5, 0.8]
    }

]

grid = GridSearchCV(
    estimator=pipe,
    param_grid= grid_params,
```

```
    cv = 5,
    scoring = "roc_auc",
    n_jobs= -1,
    verbose=1
)

grid.fit(X_train, y_train)

brazil_results = pd.DataFrame(grid.cv_results_)
brazil_best_model = grid.best_estimator_
brazil_best_params = grid.best_params_
brazil_best_cv_score = grid.best_score_



print(f"best model: {brazil_best_model}")
print(f"best params:{brazil_best_params}")
print(f"best_cv_score: {brazil_best_cv_score}")

brazil_results.head()
```

```
Fitting 5 folds for each of 20 candidates, totalling 100 fits

/Users/jamieconner/.venv/lib/python3.13/site-
packages/sklearn/linear_model/_sag.py:348: ConvergenceWarning: The max_iter was
reached which means the coef_ did not converge
  warnings.warn(

/Users/jamieconner/.venv/lib/python3.13/site-
packages/sklearn/linear_model/_sag.py:348: ConvergenceWarning: The max_iter was
reached which means the coef_ did not converge
  warnings.warn(

/Users/jamieconner/.venv/lib/python3.13/site-
packages/sklearn/linear_model/_sag.py:348: ConvergenceWarning: The max_iter was
reached which means the coef_ did not converge
  warnings.warn(

best model: Pipeline(steps=[('scaler', StandardScaler()),
                ('model',
                 LogisticRegression(C=0.1, class_weight='balanced',
                                    max_iter=3000, penalty='l1',
                                    solver='saga'))])
best params:{'model__C': 0.1, 'model__penalty': 'l1'}
best_cv_score: 0.9388488819626456
```

[4]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	\
0	3.428484	4.024024	0.007618	0.002927	
1	14.611570	18.378199	0.009292	0.003388	

```
2      20.720927     15.768454         0.007544        0.004135
3      20.759284      0.833640         0.007773        0.002272
4      10.039545      8.921824         0.007294        0.001263

   param_model__C param_model__penalty  param_model__l1_ratio  \
0            0.01                   l2                    NaN
1            0.10                   l2                    NaN
2            1.00                   l2                    NaN
3           10.00                   l2                    NaN
4            0.01                   l1                    NaN

                                      params  split0_test_score  \
0  {'model__C': 0.01, 'model__penalty': 'l2'}           0.944552
1   {'model__C': 0.1, 'model__penalty': 'l2'}           0.944554
2     {'model__C': 1, 'model__penalty': 'l2'}           0.944536
3    {'model__C': 10, 'model__penalty': 'l2'}           0.944569
4  {'model__C': 0.01, 'model__penalty': 'l1'}           0.944875

   split1_test_score  split2_test_score  split3_test_score  split4_test_score  \
0           0.935841           0.937182           0.936941           0.938798
1           0.936255           0.937467           0.936996           0.938826
2           0.935713           0.937036           0.936945           0.938812
3           0.936334           0.937465           0.936939           0.938811
4           0.935890           0.937314           0.936531           0.939022

   mean_test_score  std_test_score  rank_test_score
0         0.938663        0.003092               19
1         0.938820        0.002987                6
2         0.938608        0.003124               20
3         0.938824        0.002987                4
4         0.938726        0.003248               14
```

[5]:
```
brazil_results.sort_values("rank_test_score")
```

[5]:
```
    mean_fit_time  std_fit_time  mean_score_time  std_score_time  \
5       13.431524      6.788430         0.007157        0.002049
11       8.726645      3.869772         0.006425        0.002182
13      16.550482     15.370527         0.004723        0.000760
3       20.759284      0.833640         0.007773        0.002272
12      15.825346     14.804802         0.005554        0.002202
1       14.611570     18.378199         0.009292        0.003388
16      24.300037     10.048744         0.004660        0.001855
17      29.372587     10.165943         0.004265        0.001457
10       6.158494      3.669926         0.004756        0.001293
9        4.084306      2.572562         0.008406        0.005240
15      27.000689     18.432168         0.005779        0.002020
8        2.816283      1.193758         0.006756        0.003058
```

```
7       30.031607      2.297567         0.006925        0.001300
4       10.039545      8.921824         0.007294        0.001263
19      20.924200      3.840413         0.003596        0.000399
14      22.525303     10.146052         0.005739        0.002060
6       26.310905      8.333323         0.006233        0.001260
18      27.418156      2.860049         0.004888        0.001175
0        3.428484      4.024024         0.007618        0.002927
2       20.720927     15.768454         0.007544        0.004135

    param_model__C param_model__penalty  param_model__l1_ratio  \
5             0.10                   l1                    NaN
11            0.10           elasticnet                    0.2
13            0.10           elasticnet                    0.8
3            10.00                   l2                    NaN
12            0.10           elasticnet                    0.5
1             0.10                   l2                    NaN
16            1.00           elasticnet                    0.8
17           10.00           elasticnet                    0.2
10            0.01           elasticnet                    0.8
9             0.01           elasticnet                    0.5
15            1.00           elasticnet                    0.5
8             0.01           elasticnet                    0.2
7            10.00                   l1                    NaN
4             0.01                   l1                    NaN
19           10.00           elasticnet                    0.8
14            1.00           elasticnet                    0.2
6             1.00                   l1                    NaN
18           10.00           elasticnet                    0.5
0             0.01                   l2                    NaN
2             1.00                   l2                    NaN

                                               params  split0_test_score  \
5           {'model__C': 0.1, 'model__penalty': 'l1'}           0.944571
11  {'model__C': 0.1, 'model__l1_ratio': 0.2, 'mod…           0.944568
13  {'model__C': 0.1, 'model__l1_ratio': 0.8, 'mod…           0.944592
3            {'model__C': 10, 'model__penalty': 'l2'}           0.944569
12  {'model__C': 0.1, 'model__l1_ratio': 0.5, 'mod…           0.944568
1           {'model__C': 0.1, 'model__penalty': 'l2'}           0.944554
16  {'model__C': 1, 'model__l1_ratio': 0.8, 'model…           0.944565
17  {'model__C': 10, 'model__l1_ratio': 0.2, 'mode…           0.944513
10  {'model__C': 0.01, 'model__l1_ratio': 0.8, 'mo…           0.944843
9   {'model__C': 0.01, 'model__l1_ratio': 0.5, 'mo…           0.944726
15  {'model__C': 1, 'model__l1_ratio': 0.5, 'model…           0.944573
8   {'model__C': 0.01, 'model__l1_ratio': 0.2, 'mo…           0.944665
7            {'model__C': 10, 'model__penalty': 'l1'}           0.944551
4           {'model__C': 0.01, 'model__penalty': 'l1'}           0.944875
19  {'model__C': 10, 'model__l1_ratio': 0.8, 'mode…           0.944524
```

```
14  {'model__C': 1, 'model__l1_ratio': 0.2, 'model…           0.944553
6            {'model__C': 1, 'model__penalty': 'l1'}          0.944560
18  {'model__C': 10, 'model__l1_ratio': 0.5, 'mode…           0.944540
0          {'model__C': 0.01, 'model__penalty': 'l2'}          0.944552
2             {'model__C': 1, 'model__penalty': 'l2'}          0.944536

    split1_test_score  split2_test_score  split3_test_score  \
5            0.936323           0.937559           0.936885
11           0.936302           0.937495           0.936977
13           0.936214           0.937558           0.936915
3            0.936334           0.937465           0.936939
12           0.936222           0.937496           0.936953
1            0.936255           0.937467           0.936996
16           0.936161           0.937463           0.936935
17           0.936184           0.937513           0.936940
10           0.935872           0.937325           0.936619
9            0.935832           0.937340           0.936726
15           0.936340           0.937134           0.936940
8            0.935911           0.937310           0.936893
7            0.936317           0.937068           0.936936
4            0.935890           0.937314           0.936531
19           0.935858           0.937460           0.936937
14           0.935738           0.937451           0.936942
6            0.935734           0.937400           0.936931
18           0.935715           0.937466           0.936938
0            0.935841           0.937182           0.936941
2            0.935713           0.937036           0.936945

    split4_test_score  mean_test_score  std_test_score  rank_test_score
5            0.938907         0.938849        0.002988                1
11           0.938861         0.938841        0.002984                2
13           0.938903         0.938836        0.003011                3
3            0.938811         0.938824        0.002987                4
12           0.938878         0.938823        0.003001                5
1            0.938826         0.938820        0.002987                6
16           0.938808         0.938786        0.003016                7
17           0.938728         0.938775        0.002986                8
10           0.939197         0.938771        0.003230                9
9            0.939180         0.938761        0.003178               10
15           0.938812         0.938760        0.003020               11
8            0.938966         0.938749        0.003118               12
7            0.938810         0.938736        0.003023               13
4            0.939022         0.938726        0.003248               14
19           0.938728         0.938701        0.003054               15
14           0.938815         0.938700        0.003088               16
6            0.938817         0.938688        0.003098               17
18           0.938782         0.938688        0.003087               18
```

0	0.938798	0.938663	0.003092	19
2	0.938812	0.938608	0.003124	20

For Week 2, I compared ridge, lasso, and elastic net regularized logistic regression on the Brazil dataset using cross-validated ROC-AUC as the primary metric. Because the dataset contains over 1.7 million rows, I used a random sample to keep the grid search computationally manageable. I also used a stratified train/test split because the target variable was imbalanced. The best cross-validated model was lasso logistic regression with C = 0.1, which indicates relatively strong regularization and suggests that a sparser model generalized slightly better than the alternatives.

However, the performance differences across ridge, lasso, and elastic net were small, with mean CV scores within roughly a couple thousandths of one another. This suggests that the choice of regularization type did not dramatically change predictive performance on this dataset. Even so, lasso was still the most useful result because it provided the best CV score while also encouraging a simpler model through coefficient shrinkage and feature selection. I also tested class_weight="balanced" because of the class imbalance, but it did not materially improve ROC-AUC, suggesting that class weighting was not a major driver of performance for this particular modeling objective.

[6]:
```
df_wisc, target = load_wisconsin()
X_wisc, y_wisc = split_X_y(df_wisc, target)
```

[7]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X_wisc,
    y_wisc,
    random_state=42,
    test_size=.2,
    stratify = y_wisc
    )

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=10_000, solver="saga"))
])


grid_params = [
    {"model__penalty":["l2"], "model__C": [0.01, 0.1, 1, 10]},
    {"model__penalty":["l1"], "model__C": [0.01, 0.1, 1, 10]},

    {
      "model__penalty":["elasticnet"],
      "model__C": [0.01, 0.1, 1, 10],
      "model__l1_ratio": [0.2, 0.5, 0.8]
    }

]

grid = GridSearchCV(
```

```
    estimator=pipe,
    param_grid= grid_params,
    cv = 5,
    scoring = "roc_auc",
    n_jobs= -1,
    verbose=1
)

grid.fit(X_train, y_train)

wisc_results = pd.DataFrame(grid.cv_results_)
wisc_best_model = grid.best_estimator_
wisc_best_params = grid.best_params_
wisc_best_cv_score = grid.best_score_



print(f"best model: {wisc_best_model}")
print(f"best params:{wisc_best_params}")
print(f"best_cv_score: {wisc_best_cv_score}")

sorted_results = wisc_results.sort_values("rank_test_score")
sorted_results
```

```
Fitting 5 folds for each of 20 candidates, totalling 100 fits
best model: Pipeline(steps=[('scaler', StandardScaler()),
                ('model',
                 LogisticRegression(C=10, max_iter=10000, solver='saga'))])
best params:{'model__C': 10, 'model__penalty': 'l2'}
best_cv_score: 0.9959752321981424
```

[7]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	\
3	0.129512	0.009818	0.002109	0.001094	
19	0.284124	0.035335	0.001179	0.000035	
17	0.229983	0.012485	0.002276	0.001408	
18	0.278621	0.037073	0.001482	0.000544	
7	0.411824	0.043732	0.001258	0.000051	
2	0.030668	0.003581	0.001732	0.000859	
14	0.061324	0.002887	0.002345	0.001136	
6	0.283193	0.103907	0.001753	0.000913	
16	0.139426	0.028754	0.001840	0.000968	
15	0.082939	0.008379	0.001653	0.000594	
1	0.006162	0.000321	0.001418	0.000348	
11	0.013405	0.002945	0.002395	0.001599	
12	0.022636	0.002816	0.002080	0.001509	
13	0.039064	0.003158	0.001516	0.000598	
5	0.122866	0.019517	0.001453	0.000384	

```
0        0.003505      0.000191         0.001102        0.000055
8        0.006423      0.003126         0.002396        0.001240
9        0.006230      0.001710         0.001533        0.000630
10       0.007540      0.001543         0.002050        0.001189
4        0.020397      0.005686         0.001775        0.000776

    param_model__C param_model__penalty  param_model__l1_ratio  \
3           10.00                    l2                    NaN
19          10.00           elasticnet                    0.8
17          10.00           elasticnet                    0.2
18          10.00           elasticnet                    0.5
7           10.00                    l1                    NaN
2            1.00                    l2                    NaN
14           1.00           elasticnet                    0.2
6            1.00                    l1                    NaN
16           1.00           elasticnet                    0.8
15           1.00           elasticnet                    0.5
1            0.10                    l2                    NaN
11           0.10           elasticnet                    0.2
12           0.10           elasticnet                    0.5
13           0.10           elasticnet                    0.8
5            0.10                    l1                    NaN
0            0.01                    l2                    NaN
8            0.01           elasticnet                    0.2
9            0.01           elasticnet                    0.5
10           0.01           elasticnet                    0.8
4            0.01                    l1                    NaN

                                              params  split0_test_score  \
3           {'model__C': 10, 'model__penalty': 'l2'}           0.997936
19  {'model__C': 10, 'model__l1_ratio': 0.8, 'mode…           0.997936
17  {'model__C': 10, 'model__l1_ratio': 0.2, 'mode…           0.997936
18  {'model__C': 10, 'model__l1_ratio': 0.5, 'mode…           0.997420
7           {'model__C': 10, 'model__penalty': 'l1'}           0.997420
2            {'model__C': 1, 'model__penalty': 'l2'}           0.998452
14  {'model__C': 1, 'model__l1_ratio': 0.2, 'model…           0.998452
6            {'model__C': 1, 'model__penalty': 'l1'}           0.998452
16  {'model__C': 1, 'model__l1_ratio': 0.8, 'model…           0.998452
15  {'model__C': 1, 'model__l1_ratio': 0.5, 'model…           0.998452
1          {'model__C': 0.1, 'model__penalty': 'l2'}           0.997936
11  {'model__C': 0.1, 'model__l1_ratio': 0.2, 'mod…           0.997936
12  {'model__C': 0.1, 'model__l1_ratio': 0.5, 'mod…           0.997936
13  {'model__C': 0.1, 'model__l1_ratio': 0.8, 'mod…           0.998452
5           {'model__C': 0.1, 'model__penalty': 'l1'}           0.998452
0          {'model__C': 0.01, 'model__penalty': 'l2'}           0.995356
8   {'model__C': 0.01, 'model__l1_ratio': 0.2, 'mo…           0.995872
9   {'model__C': 0.01, 'model__l1_ratio': 0.5, 'mo…           0.996904
```

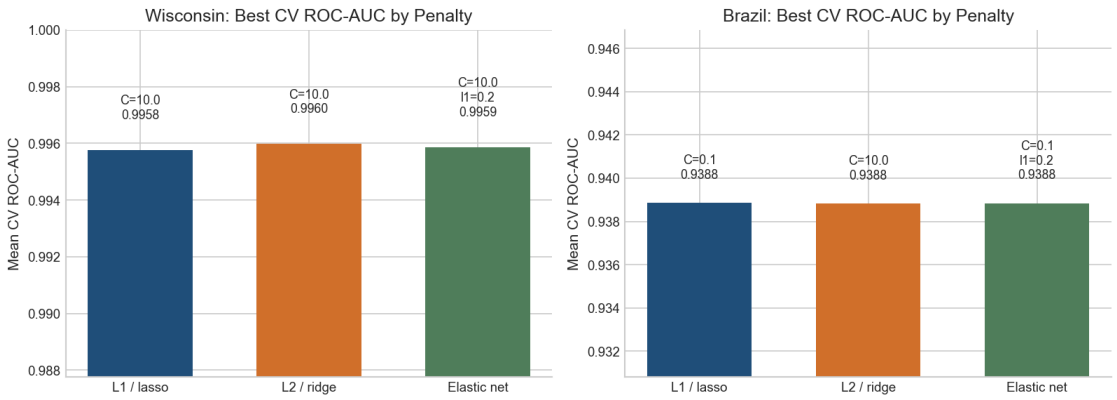
```
10  {'model__C': 0.01, 'model__l1_ratio': 0.8, 'mo…           0.996904
4          {'model__C': 0.01, 'model__penalty': 'l1'}           0.996388

    split1_test_score  split2_test_score  split3_test_score  \
3            1.000000           0.987616           1.000000
19           1.000000           0.987616           1.000000
17           1.000000           0.987616           1.000000
18           1.000000           0.987616           1.000000
7            1.000000           0.987100           1.000000
2            1.000000           0.984004           0.998968
14           1.000000           0.984004           0.998968
6            1.000000           0.984004           0.998452
16           1.000000           0.982972           0.998452
15           1.000000           0.982456           0.998968
1            1.000000           0.982456           0.998968
11           1.000000           0.981940           0.998452
12           1.000000           0.978844           0.998968
13           1.000000           0.979360           0.999484
5            0.999484           0.980392           1.000000
0            1.000000           0.976264           0.997420
8            1.000000           0.973168           0.997420
9            0.998968           0.966460           0.997420
10           0.997936           0.968008           0.997936
4            0.995356           0.962332           0.995356

    split4_test_score  mean_test_score  std_test_score  rank_test_score
3            0.994324         0.995975        0.004666                1
19           0.993808         0.995872        0.004707                2
17           0.993808         0.995872        0.004707                2
18           0.993808         0.995769        0.004666                4
7            0.994324         0.995769        0.004812                5
2            0.994324         0.995150        0.005898                6
14           0.994324         0.995150        0.005898                6
6            0.993292         0.994840        0.005874                8
16           0.993808         0.994737        0.006238                9
15           0.993808         0.994737        0.006497               10
1            0.990196         0.993911        0.006691               11
11           0.986584         0.992982        0.007302               12
12           0.981424         0.991434        0.009286               13
13           0.976264         0.990712        0.010590               14
5            0.975232         0.990712        0.010670               15
0            0.983488         0.990506        0.009094               16
8            0.976264         0.988545        0.011410               17
9            0.965944         0.985139        0.015478               18
10           0.962332         0.984623        0.015989               19
4            0.954592         0.980805        0.018410               20
```

Compared with the unregularized or baseline logistic model from Week 1, regularized logistic re-

gression produced very similar performance. The best Week 2 model used L2 regularization with C=10, but several L1 and elastic net models had nearly identical cross-validation scores. This suggests that the Wisconsin data are already clean and strongly separable, so regularization mainly provides stability rather than a major performance improvement. The main value of regularization here is not boosting accuracy, but controlling coefficient size and reducing risk from correlated tumor-measurement features.

[8]:
```
import matplotlib.pyplot as plt

plt.style.use("seaborn-v0_8-whitegrid")
plt.rcParams.update({
    "figure.dpi": 150,
    "savefig.dpi": 300,
    "font.size": 10,
    "axes.titlesize": 12,
    "axes.labelsize": 10,
    "xtick.labelsize": 9,
    "ytick.labelsize": 9,
    "legend.fontsize": 9,
})
FIGURE_DIR = PROJECT_ROOT / "outputs" / "figures" / "milestone1"
FIGURE_DIR.mkdir(parents=True, exist_ok=True)

def summarize_regularization_results(results_df):
    summary = results_df.copy()
    summary["penalty_family"] = summary["param_model__penalty"].astype(str)
    summary["penalty_label"] = summary["penalty_family"].map({
        "l1": "L1 / lasso",
        "l2": "L2 / ridge",
        "elasticnet": "Elastic net",
    })
    best_by_penalty = (
        summary.sort_values(["penalty_family", "mean_test_score"],␣
 ↪ascending=[True, False])
        .groupby("penalty_family", as_index=False)
        .first()
    )
    return best_by_penalty[["penalty_family", "penalty_label",␣
 ↪"param_model__C", "param_model__l1_ratio", "mean_test_score"]]

brazil_plot_df = summarize_regularization_results(brazil_results)
wisc_plot_df = summarize_regularization_results(wisc_results)
order = ["l1", "l2", "elasticnet"]
plot_frames = [
    ("Wisconsin", wisc_plot_df.set_index("penalty_family").loc[order].
 ↪reset_index()),
```

```
    ("Brazil", brazil_plot_df.set_index("penalty_family").loc[order].
 ↪reset_index()),
]
colors = ["#1F4E79", "#D06F2A", "#4F7D5A"]

fig, axes = plt.subplots(1, 2, figsize=(10.8, 4.3), sharey=False)
for ax, (dataset_name, plot_df) in zip(axes, plot_frames):
    bars = ax.bar(plot_df["penalty_label"], plot_df["mean_test_score"],␣
 ↪color=colors, width=0.62)
    ax.set_title(f"{dataset_name}: Best CV ROC-AUC by Penalty")
    ax.set_ylabel("Mean CV ROC-AUC")
    y_min = max(0, plot_df["mean_test_score"].min() - 0.008)
    y_max = min(1.0, plot_df["mean_test_score"].max() + 0.008)
    ax.set_ylim(y_min, y_max)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    for bar, (_, row) in zip(bars, plot_df.iterrows()):
        ratio = row["param_model__l1_ratio"]
        ratio_text = "" if ratio != ratio else f"\nl1={ratio}"
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            row["mean_test_score"] + 0.001,
            f"C={row['param_model__C']}{ratio_text}\n{row['mean_test_score']:.
 ↪4f}",
            ha="center",
            va="bottom",
            fontsize=8.5,
        )

fig.suptitle("Week 2 Regularized Logistic Regression Comparison", fontsize=14,␣
 ↪weight="bold", y=1.02)
fig.tight_layout()
fig.savefig(FIGURE_DIR / "fig02_regularization_cv_roc_auc.png",␣
 ↪bbox_inches="tight", facecolor="white")
plt.show()
```


Week 2 Regularized Logistic Regression Comparison
Wisconsin: Best CV ROC-AUC by Penalty
Mean CV ROC-AUC
1.000
0.998
0.996
0.994
0.992
0.990
0.988
C=10.0
0.9958
C=10.0
0.9960
C=10.0
l1=0.2
0.9959
L1 / lasso
L2 / ridge
Elastic net
Brazil: Best CV ROC-AUC by Penalty
Mean CV ROC-AUC
0.946
0.944
0.942
0.940
0.938
0.936
0.934
0.932
C=0.1
0.9388
C=10.0
0.9388
C=0.1
l1=0.2
0.9388
L1 / lasso
L2 / ridge
Elastic net

03_week3_feature_selection_pcr_pls

June 28, 2026

Each week, you will apply the concepts of that week to your Integrated Capstone Project's dataset. In preparation for Milestone One, create a Jupyter Notebook (similar to in Module B, Semester Two) that illustrates these lessons. There are no specific questions to answer in your Jupyter Notebook files in this course; your general goal is to analyze your data, using the methods you have learned about in this course and in this program, and draw interesting conclusions.

For Week 3, include concepts such as linear regression with forward and backward selection, PCR, and PLSR. Complete your Jupyter Notebook homework by 11:59 pm ET on Sunday.

In Week 7, you will compile your findings from your Jupyter Notebook homework into your Milestone One assignment for grading. For full instructions and the rubric for Milestone One, refer to the following link.

[1]:
```
from pathlib import Path
import sys

PROJECT_ROOT = Path.cwd().resolve().parents[1]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.append(str(PROJECT_ROOT))

print(PROJECT_ROOT)
```

```
/Users/jamieconner/projects/bu/dx799_01/mod_c_capstone
```

[2]:
```
import pandas as pd
from src.load_data import load_wisconsin, load_brazil, split_X_y
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.decomposition import PCA
from sklearn.cross_decomposition import PLSRegression
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.model_selection import GridSearchCV, StratifiedKFold,␣
 ↪train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
 ↪f1_score, roc_auc_score
```

[3]:
```
df_wisc, target = load_wisconsin()
X_wisc, y_wisc = split_X_y(df_wisc, target)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_wisc, y_wisc, test_size=0.2, random_state=42, stratify=y_wisc
)
```

[4]:
```
##### PCA works naturally in a pipeline because it is an X-only transformer: it␣
 ↪fits component directions from X and returns transformed X components.
##### Logistic regression then uses those transformed X components as the final␣
 ↪classifier.
##### PLSR is different because it uses both X and y when fitting its␣
 ↪target-informed components.
##### I used a small class wrapper to make PLSR behave like a␣
 ↪pipeline-compatible transformer: fit with X and y, then return only the␣
 ↪transformed X scores.
##### This lets forward selection, PCR, and PLSR use the same pipeline/
 ↪GridSearchCV/CV workflow.
```

[5]:
```
from sklearn.base import BaseEstimator, TransformerMixin


class PLSTransformer(BaseEstimator, TransformerMixin):
    def __init__(self, n_components=2):
        self.n_components = n_components

    def fit(self, X, y):
        self.pls_ = PLSRegression(n_components=self.n_components)
        self.pls_.fit(X, y)
        return self

    def transform(self, X):
        return self.pls_.transform(X)
```

[6]:
```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

base_logreg = LogisticRegression(
    max_iter=10000,
    solver="saga",
    random_state=42
)

pipelines = {
    "forward_selection": Pipeline([
        ("scaler", StandardScaler()),
        ("selector", SequentialFeatureSelector(
            estimator=LogisticRegression(max_iter=10000, solver="saga",␣
 ↪random_state=42),
            direction="forward",
```

```
            scoring="roc_auc",
            cv=3,
            n_jobs=-1
        )),
        ("model", base_logreg)
    ]),

    "pcr": Pipeline([
        ("scaler", StandardScaler()),
        ("pca", PCA()),
        ("model", base_logreg)
    ]),

    "plsr": Pipeline([
        ("scaler", StandardScaler()),
        ("pls", PLSTransformer()),
        ("model", base_logreg)
    ])
}

param_grids = {
    "forward_selection": {
        "selector__n_features_to_select": [5, 10, 15],
        "model__C": [0.1, 1, 10],
        "model__penalty": ["l2"]
    },

    "pcr": {
        "pca__n_components": [2, 3, 5, 10, 15],
        "model__C": [0.1, 1, 10],
        "model__penalty": ["l2"]
    },

    "plsr": {
        "pls__n_components": [2, 3, 5, 10, 15],
        "model__C": [0.1, 1, 10],
        "model__penalty": ["l2"]
    }
}
```

[7]:
```
wisc_results = []
wisc_best_models = {}

for name, pipe in pipelines.items():
    print(f"\nRunning {name}...")

    search = GridSearchCV(
```

```
        estimator=pipe,
        param_grid=param_grids[name],
        scoring="roc_auc",
        cv=cv,
        n_jobs=-1,
        verbose=1
    )

    search.fit(X_train, y_train)

    best_model = search.best_estimator_
    wisc_best_models[name] = best_model

    y_pred = best_model.predict(X_test)
    y_proba = best_model.predict_proba(X_test)[:, 1]

    wisc_results.append({
        "model": name,
        "best_cv_roc_auc": search.best_score_,
        "test_accuracy": accuracy_score(y_test, y_pred),
        "test_precision": precision_score(y_test, y_pred, zero_division=0),
        "test_recall": recall_score(y_test, y_pred, zero_division=0),
        "test_f1": f1_score(y_test, y_pred, zero_division=0),
        "test_roc_auc": roc_auc_score(y_test, y_proba),
        "best_params": search.best_params_
    })

wisc_results_df = pd.DataFrame(wisc_results).sort_values("best_cv_roc_auc",␣
 ↪ascending=False)
wisc_results_df
```

```
Running forward_selection…
Fitting 5 folds for each of 9 candidates, totalling 45 fits


Running pcr…
Fitting 5 folds for each of 15 candidates, totalling 75 fits

Running plsr…
Fitting 5 folds for each of 15 candidates, totalling 75 fits
```

[7]:

	model	best_cv_roc_auc	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc
2	plsr	0.995769	0.991228	1.000000	0.976744	0.988235	0.997354
1	pcr	0.995150	0.982456	1.000000	0.953488	0.976190	0.997354
0	forward_selection	0.994840	0.956140	0.974359	0.883721	0.926829	0.994709

```
2     0.976190  0.987952      0.997354
1     0.952381  0.975610      0.997024
0     0.904762  0.938272      0.994709

                                              best_params
2  {'model__C': 1, 'model__penalty': 'l2', 'pls__…
1  {'model__C': 1, 'model__penalty': 'l2', 'pca__…
0  {'model__C': 1, 'model__penalty': 'l2', 'selec…
```

0.0.1 Wisconsin summary

On Wisconsin, all three Week 3 approaches performed well, which is consistent with the rest of the project: the dataset is small, clean, and already highly separable. Forward selection remained the most interpretable because it kept original predictors, while PCR and PLSR offered alternative ways to handle correlation through lower-dimensional components.

0.0.2 Brazil comparison

To satisfy Milestone 1 breadth, I also applied forward selection, PCR, and PLSR to the Brazil dataset. I used an 8,000-row sample here to keep the notebook runnable while still testing the same Week 3 ideas on the larger, more imbalanced dataset.

[8]:
```
df_brazil, target_brazil = load_brazil(sample_size=8000, random_state=42)
X_brazil, y_brazil = split_X_y(df_brazil, target_brazil)

X_train_brazil, X_test_brazil, y_train_brazil, y_test_brazil = train_test_split(
    X_brazil, y_brazil, test_size=0.2, random_state=42, stratify=y_brazil
)

brazil_results = []
brazil_best_models = {}

for name, pipe in pipelines.items():
    print(f"\nRunning Brazil {name}...")

    search = GridSearchCV(
        estimator=pipe,
        param_grid=param_grids[name],
        scoring="roc_auc",
        cv=cv,
        n_jobs=-1,
        verbose=1
    )

    search.fit(X_train_brazil, y_train_brazil)

    best_model = search.best_estimator_
    brazil_best_models[name] = best_model
```

```
    y_pred = best_model.predict(X_test_brazil)
    y_proba = best_model.predict_proba(X_test_brazil)[:, 1]

    brazil_results.append({
        "model": name,
        "best_cv_roc_auc": search.best_score_,
        "test_accuracy": accuracy_score(y_test_brazil, y_pred),
        "test_precision": precision_score(y_test_brazil, y_pred,␣
 ↪zero_division=0),
        "test_recall": recall_score(y_test_brazil, y_pred, zero_division=0),
        "test_f1": f1_score(y_test_brazil, y_pred, zero_division=0),
        "test_roc_auc": roc_auc_score(y_test_brazil, y_proba),
        "best_params": search.best_params_
    })

brazil_results_df = pd.DataFrame(brazil_results).sort_values("best_cv_roc_auc",␣
 ↪ascending=False)
brazil_results_df
```

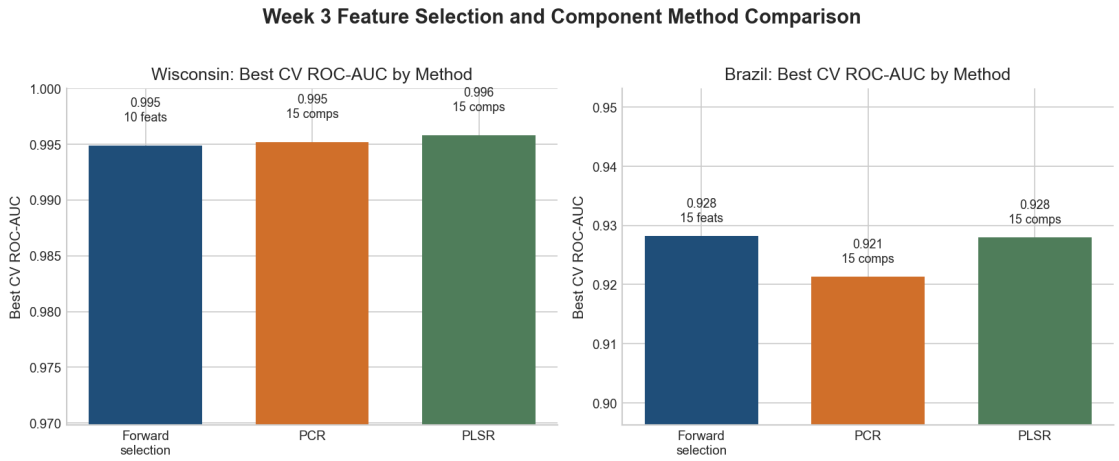
```
Running Brazil forward_selection…
Fitting 5 folds for each of 9 candidates, totalling 45 fits


Running Brazil pcr…
Fitting 5 folds for each of 15 candidates, totalling 75 fits


Running Brazil plsr…
Fitting 5 folds for each of 15 candidates, totalling 75 fits
```

[8]:

	model	best_cv_roc_auc	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc	best_params
0	forward_selection	0.928155	0.944375	0.706897	0.362832	0.479532	0.913400	{'model__C': 10, 'model__penalty': 'l2', 'sele…
2	plsr	0.927942	0.942500	0.684211	0.345133	0.458824	0.917968	{'model__C': 0.1, 'model__penalty': 'l2', 'pls…
1	pcr	0.921272	0.939375	0.637931	0.327434	0.432749	0.906743	{'model__C': 10, 'model__penalty': 'l2', 'pca_…

[9]:
```
import matplotlib.pyplot as plt

plt.style.use("seaborn-v0_8-whitegrid")
plt.rcParams.update({
    "figure.dpi": 150,
    "savefig.dpi": 300,
    "font.size": 10,
    "axes.titlesize": 12,
    "axes.labelsize": 10,
    "xtick.labelsize": 9,
    "ytick.labelsize": 9,
    "legend.fontsize": 9,
})
FIGURE_DIR = PROJECT_ROOT / "outputs" / "figures" / "milestone1"
FIGURE_DIR.mkdir(parents=True, exist_ok=True)

def extract_model_size(best_params):
    if "selector__n_features_to_select" in best_params:
        return f"{best_params['selector__n_features_to_select']} feats"
    if "pca__n_components" in best_params:
        return f"{best_params['pca__n_components']} comps"
    if "pls__n_components" in best_params:
        return f"{best_params['pls__n_components']} comps"
    return ""

plot_order = ["forward_selection", "pcr", "plsr"]
pretty_names = {
    "forward_selection": "Forward\nselection",
    "pcr": "PCR",
    "plsr": "PLSR",
}
plot_frames = [
    ("Wisconsin", wisc_results_df.set_index("model").loc[plot_order].
 ↪reset_index()),
    ("Brazil", brazil_results_df.set_index("model").loc[plot_order].
 ↪reset_index()),
]
colors = ["#1F4E79", "#D06F2A", "#4F7D5A"]

fig, axes = plt.subplots(1, 2, figsize=(11, 4.4), sharey=False)
for ax, (dataset_name, plot_df) in zip(axes, plot_frames):
    labels = [pretty_names[name] for name in plot_df["model"]]
    bars = ax.bar(labels, plot_df["best_cv_roc_auc"], color=colors, width=0.68)
    ax.set_title(f"{dataset_name}: Best CV ROC-AUC by Method")
    ax.set_ylabel("Best CV ROC-AUC")
    y_min = max(0, plot_df["best_cv_roc_auc"].min() - 0.025)
    y_max = min(1.0, plot_df["best_cv_roc_auc"].max() + 0.025)
```

```
    ax.set_ylim(y_min, y_max)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    for bar, (_, row) in zip(bars, plot_df.iterrows()):
        size_label = extract_model_size(row["best_params"])
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            row["best_cv_roc_auc"] + 0.002,
            f"{row['best_cv_roc_auc']:.3f}\n{size_label}",
            ha="center",
            va="bottom",
            fontsize=8.5,
        )

fig.suptitle("Week 3 Feature Selection and Component Method Comparison",␣
 ↪fontsize=14, weight="bold", y=1.02)
fig.tight_layout()
fig.savefig(FIGURE_DIR / "fig03_feature_selection_pcr_plsr.png",␣
 ↪bbox_inches="tight", facecolor="white")
plt.show()
```



0.0.3 Combined Week 3 summary

Across both datasets, Week 3 reinforced the tradeoff between interpretability and component-based dimensionality reduction. Forward selection stayed the most interpretable because it selected original features directly, while PCR and PLSR compressed the predictors into a smaller number of components. On Wisconsin, all three methods were very strong and close together, which makes sense for a clean and highly separable dataset.

Brazil was more informative because the differences were easier to see. The best cross-validated ROC-AUC came from the stronger component-based approaches, while forward selection remained easier to explain in original-variable terms. That makes Week 3 useful for the Milestone 1 breadth

section: component methods can help generalization when the feature space is larger and more correlated, but feature selection still has an advantage when interpretability matters.

04_week4_logistic_scaling_thresholds

June 28, 2026

Each week, you will apply the concepts of that week to your Integrated Capstone Project's dataset. In preparation for Milestone One, create a Jupyter Notebook (similar to in Module B, semester two) that illustrates these lessons. There are no specific questions to answer in your Jupyter Notebook files in this course; your general goal is to analyze your data using the methods you have learned about in this course and in this program and draw interesting conclusions.

For Week 4, include concepts such as logistic regression and feature scaling. This homework should be submitted for peer review in the assignment titled 4.3 Peer Review: Week 4 Jupyter Notebook. Complete and submit your Jupyter Notebook homework by 11:59pm ET on Sunday.

In Week 7, you will compile your findings from your Jupyter Notebook homework into your Milestone One assignment for grading. For full instructions and the rubric for Milestone One, refer to the following link.

[1]:
```
from pathlib import Path
import sys

PROJECT_ROOT = Path.cwd().resolve().parents[1]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.append(str(PROJECT_ROOT))

print(PROJECT_ROOT)
```

```
/Users/jamieconner/projects/bu/dx799_01/mod_c_capstone
```

0.0.1 Imports

[2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV,␣
 ↪StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
```

```
    recall_score,
    f1_score,
    roc_auc_score,
    average_precision_score,
    confusion_matrix,
    classification_report,
    precision_recall_curve
)

from src.load_data import load_wisconsin, load_brazil, split_X_y
```

[3]:
```
def evaluate_binary_model(name, y_true, y_pred, y_score=None):
    row = {
        "model": name,
        "accuracy": accuracy_score(y_true, y_pred),
        "precision": precision_score(y_true, y_pred, zero_division=0),
        "recall": recall_score(y_true, y_pred, zero_division=0),
        "f1": f1_score(y_true, y_pred, zero_division=0),
    }

    if y_score is not None:
        row["roc_auc"] = roc_auc_score(y_true, y_score)
        row["pr_auc"] = average_precision_score(y_true, y_score)

    return row
```

[4]:
```
df_w, target_w = load_wisconsin()
X_w, y_w = split_X_y(df_w, target_w)



Xw_train, Xw_test, yw_train, yw_test = train_test_split(
    X_w,
    y_w,
    test_size=0.2,
    random_state=42,
    stratify=y_w
)
```

[5]:
```
wisconsin_models = {
    "Wisconsin Logistic Unscaled": Pipeline([
        ("model", LogisticRegression(max_iter=10000, random_state=42))
    ]),

    "Wisconsin Logistic Scaled": Pipeline([
        ("scaler", StandardScaler()),
        ("model", LogisticRegression(max_iter=10000, random_state=42))
```

```
    ])
}

wisconsin_results = []

for name, model in wisconsin_models.items():
    model.fit(Xw_train, yw_train)

    y_pred = model.predict(Xw_test)
    y_score = model.predict_proba(Xw_test)[:, 1]

    wisconsin_results.append(evaluate_binary_model(name, yw_test, y_pred,␣
 ↪y_score))

wisconsin_results_df = pd.DataFrame(wisconsin_results)
wisconsin_results_df
```

[5]:

	model	accuracy	precision	recall	f1	roc_auc	pr_auc
0	Wisconsin Logistic Unscaled	0.938596	0.972973	0.857143	0.911392	0.992394	0.986931
1	Wisconsin Logistic Scaled	0.964912	0.975000	0.928571	0.951220	0.996032	0.994274

[6]:
```
df_b, target_b = load_brazil(sample_size=100000, random_state=42)
X_b, y_b = split_X_y(df_b, target_b)

Xb_train, Xb_test, yb_train, yb_test = train_test_split(
    X_b,
    y_b,
    test_size=0.2,
    random_state=42,
    stratify=y_b
)
```

[7]:
```
brazil_models = {
    "Brazil Logistic Scaled": Pipeline([
        ("scaler", StandardScaler()),
        ("model", LogisticRegression(max_iter=10000, random_state=42))
    ]),

    "Brazil Logistic Scaled Balanced": Pipeline([
        ("scaler", StandardScaler()),
        ("model", LogisticRegression(
            max_iter=10000,
            random_state=42,
```

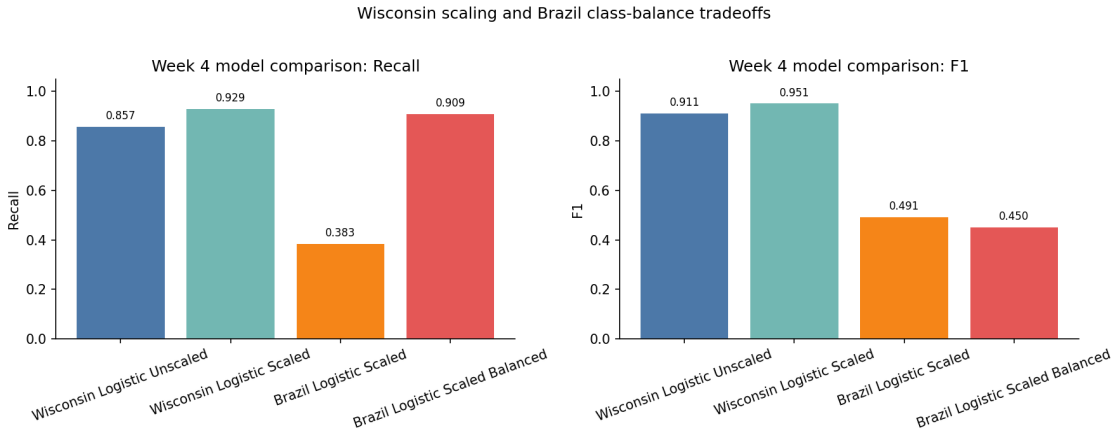
```
            class_weight="balanced"
        ))
    ])
}

brazil_results = []

for name, model in brazil_models.items():
    model.fit(Xb_train, yb_train)

    y_pred = model.predict(Xb_test)
    y_score = model.predict_proba(Xb_test)[:, 1]

    brazil_results.append(evaluate_binary_model(name, yb_test, y_pred, y_score))

brazil_results_df = pd.DataFrame(brazil_results)
brazil_results_df
```

[7]:

	model	accuracy	precision	recall	f1	roc_auc	pr_auc
0	Brazil Logistic Scaled	0.94425	0.682741	0.383464	0.491100	0.938624	0.498608
1	Brazil Logistic Scaled Balanced	0.84420	0.299085	0.908767	0.450053	0.940950	0.479120

[8]:
```
comparison_df = pd.concat([wisconsin_results_df, brazil_results_df],␣
 ↪ignore_index=True)
comparison_order = [
    "Wisconsin Logistic Unscaled",
    "Wisconsin Logistic Scaled",
    "Brazil Logistic Scaled",
    "Brazil Logistic Scaled Balanced",
]
comparison_df = comparison_df.set_index("model").loc[comparison_order].
 ↪reset_index()

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5), dpi=150)
metrics = [("recall", "Recall"), ("f1", "F1")]
colors = ["#4C78A8", "#72B7B2", "#F58518", "#E45756"]

for ax, (metric_key, metric_label) in zip(axes, metrics):
    bars = ax.bar(comparison_df["model"], comparison_df[metric_key],␣
 ↪color=colors)
    ax.set_title(f"Week 4 model comparison: {metric_label}")
    ax.set_ylabel(metric_label)
    ax.set_ylim(0, 1.05)
```

```
    ax.tick_params(axis="x", rotation=20)
    for bar, value in zip(bars, comparison_df[metric_key]):
        ax.text(bar.get_x() + bar.get_width() / 2, value + 0.02, f"{value:.
↪3f}", ha="center", va="bottom", fontsize=8)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)

fig.suptitle("Wisconsin scaling and Brazil class-balance tradeoffs", y=1.02)
fig.tight_layout()
plt.show()
```



[9]:
```
best_brazil_logreg = brazil_models["Brazil Logistic Scaled Balanced"]
best_brazil_logreg.fit(Xb_train, yb_train)

yb_score = best_brazil_logreg.predict_proba(Xb_test)[:, 1]
```

[10]:
```
thresholds = np.arange(0.05, 0.96, 0.05)

threshold_rows = []

for threshold in thresholds:
    y_pred_thresh = (yb_score >= threshold).astype(int)

    threshold_rows.append({
        "threshold": threshold,
        "precision": precision_score(yb_test, y_pred_thresh, zero_division=0),
        "recall": recall_score(yb_test, y_pred_thresh, zero_division=0),
        "f1": f1_score(yb_test, y_pred_thresh, zero_division=0),
        "accuracy": accuracy_score(yb_test, y_pred_thresh),
    })
```

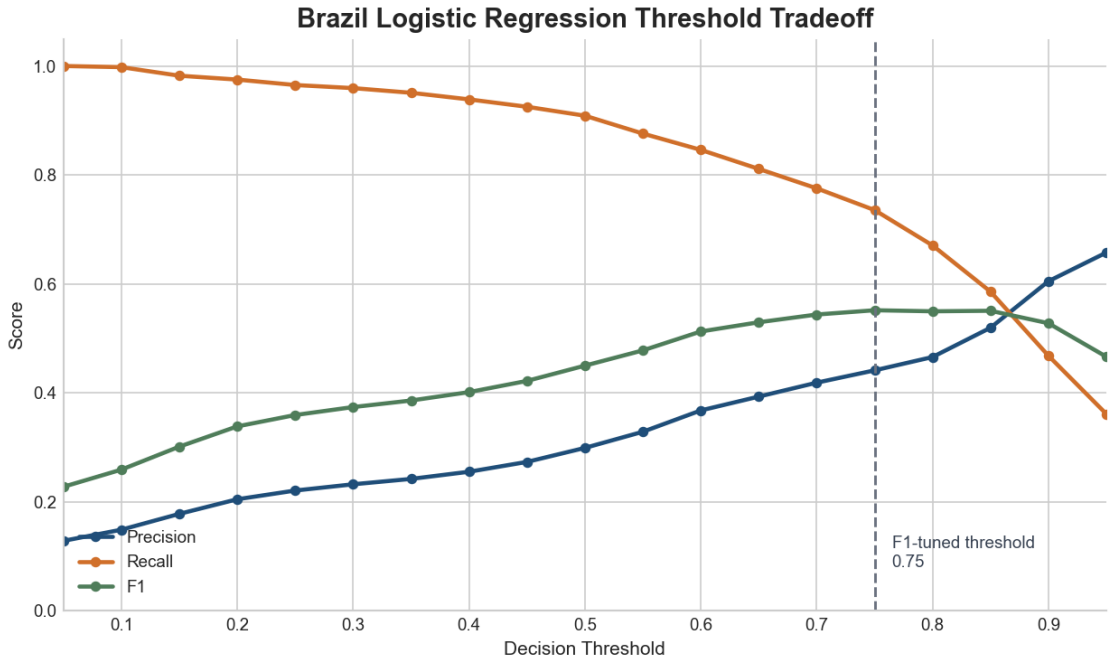
```
threshold_df = pd.DataFrame(threshold_rows)
threshold_df.head()
```

[10]:

	threshold	precision	recall	f1	accuracy
0	0.05	0.128492	1.000000	0.227723	0.52420
1	0.10	0.148968	0.997862	0.259235	0.59995
2	0.15	0.178013	0.982181	0.301400	0.68060
3	0.20	0.204852	0.975053	0.338572	0.73275
4	0.25	0.220845	0.965075	0.359437	0.75870

[11]:
```
import matplotlib.pyplot as plt

plt.style.use("seaborn-v0_8-whitegrid")
plt.rcParams.update({
    "figure.dpi": 150,
    "savefig.dpi": 300,
    "font.size": 10,
    "axes.titlesize": 12,
    "axes.labelsize": 10,
    "xtick.labelsize": 9,
    "ytick.labelsize": 9,
    "legend.fontsize": 9,
})
FIGURE_DIR = PROJECT_ROOT / "outputs" / "figures" / "milestone1"
FIGURE_DIR.mkdir(parents=True, exist_ok=True)

best_f1_row = threshold_df.loc[threshold_df["f1"].idxmax()]

fig, ax = plt.subplots(figsize=(8.4, 5.1))

line_specs = [
    ("precision", "Precision", "#1F4E79"),
    ("recall", "Recall", "#D06F2A"),
    ("f1", "F1", "#4F7D5A"),
]
for metric, label, color in line_specs:
    ax.plot(threshold_df["threshold"], threshold_df[metric], marker="o",␣
 ↪linewidth=2.2, markersize=4.5, label=label, color=color)

best_threshold = float(best_f1_row["threshold"])
ax.axvline(best_threshold, color="#6B7280", linestyle="--", linewidth=1.4)
ax.text(best_threshold + 0.015, 0.08, f"F1-tuned threshold\n{best_threshold:.
 ↪2f}", color="#374151", fontsize=9)

ax.set_title("Brazil Logistic Regression Threshold Tradeoff", fontsize=14,␣
 ↪weight="bold")
ax.set_xlabel("Decision Threshold")
```

```
ax.set_ylabel("Score")
ax.set_ylim(0, 1.05)
ax.set_xlim(threshold_df["threshold"].min(), threshold_df["threshold"].max())
ax.legend(frameon=False, loc="lower left")
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

fig.tight_layout()
fig.savefig(FIGURE_DIR / "fig04_brazil_threshold_tradeoff.png",␣
 ↪bbox_inches="tight", facecolor="white")
plt.show()
```



[12]:
```
best_f1_row = threshold_df.loc[threshold_df["f1"].idxmax()]
best_f1_row
```

[12]:
```
threshold    0.750000
precision    0.441403
recall       0.735567
f1           0.551724
accuracy     0.916150
Name: 14, dtype: float64
```

[13]:
```
recall_target =  0.735567

high_recall_options = threshold_df[threshold_df["recall"] >= recall_target]
```

```
if len(high_recall_options) > 0:
    best_high_recall = high_recall_options.sort_values("precision",␣
 ↪ascending=False).iloc[0]
    print(best_high_recall)
else:
    print("No threshold achieved the recall target.")
```

```
threshold    0.700000
precision    0.418784
recall       0.775481
f1           0.543864
accuracy     0.908750
Name: 13, dtype: float64
```

[14]:
```
default_pred = (yb_score >= 0.50).astype(int)

best_threshold = best_f1_row["threshold"]
tuned_pred = (yb_score >= best_threshold).astype(int)

print("Default threshold confusion matrix:")
print(confusion_matrix(yb_test, default_pred))

print("\nTuned threshold confusion matrix:")
print(confusion_matrix(yb_test, tuned_pred))

print("\nDefault threshold report:")
print(classification_report(yb_test, default_pred, zero_division=0))

print("\nTuned threshold report:")
print(classification_report(yb_test, tuned_pred, zero_division=0))
```

```
Default threshold confusion matrix:
[[15609  2988]
 [  128  1275]]

Tuned threshold confusion matrix:
[[17291  1306]
 [  371  1032]]

Default threshold report:
              precision    recall  f1-score   support

           0       0.99      0.84      0.91     18597
           1       0.30      0.91      0.45      1403

    accuracy                           0.84     20000
   macro avg       0.65      0.87      0.68     20000
```

```
weighted avg       0.94      0.84      0.88     20000


Tuned threshold report:
              precision    recall  f1-score   support

           0       0.98      0.93      0.95     18597
           1       0.44      0.74      0.55      1403

    accuracy                           0.92     20000
   macro avg       0.71      0.83      0.75     20000
weighted avg       0.94      0.92      0.93     20000
```

This Week 4 notebook applied logistic regression and feature scaling to the Wisconsin and Brazil cancer datasets in preparation for Milestone 1. For Wisconsin, scaling improved logistic regression performance, raising accuracy from about 0.94 to 0.96 and recall from about 0.86 to 0.93. This supports the earlier conclusion that Wisconsin is a clean, highly separable benchmark dataset, but also shows that scaling still matters for logistic regression because the model is sensitive to feature scale.

The Brazil analysis was more important for the overall Milestone 1 story because it showed how model evaluation changes under class imbalance. The standard scaled logistic regression had higher accuracy, but the balanced model greatly improved recall for the positive class. Threshold tuning then showed the tradeoff clearly: the default threshold caught more positive cases, while the tuned threshold improved precision, F1, and overall accuracy at the cost of lower recall. This directly supports the project's diagnostic workflow theme: model quality should not be judged by accuracy alone, especially when false negatives carry clinical risk.

05_week5_svm

June 28, 2026

Each week, you will apply the concepts of that week to your Integrated Capstone Project

For Week 5, the focus is support vector machines. This notebook compares linear and rad

[1]:
```
from pathlib import Path
import sys

PROJECT_ROOT = Path.cwd().resolve().parents[1]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.append(str(PROJECT_ROOT))

PROJECT_ROOT
```

[1]:
```
PosixPath('/Users/jamieconner/projects/bu/dx799_01/mod_c_capstone')
```

0.1 Imports

[2]:
```
import warnings

import matplotlib.pyplot as plt
import pandas as pd
from sklearn.exceptions import ConvergenceWarning
from sklearn.metrics import (
    accuracy_score,
    average_precision_score,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
)
from sklearn.model_selection import GridSearchCV, StratifiedKFold,␣
 ↪train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

from src.load_data import load_brazil, load_wisconsin, split_X_y
```

```
warnings.filterwarnings("ignore", category=ConvergenceWarning)
plt.style.use("seaborn-v0_8-whitegrid")
pd.set_option("display.max_colwidth", 80)
RANDOM_STATE = 42
```

Wisconsin dataset

Wisconsin is small, clean, and pretty separable, so I wanted to see whether an RBF ker

[3]:
```
df_w, target_w = load_wisconsin()
X_w, y_w = split_X_y(df_w, target_w)

Xw_train, Xw_test, yw_train, yw_test = train_test_split(
    X_w,
    y_w,
    test_size=0.2,
    stratify=y_w,
    random_state=RANDOM_STATE,
)

print("Wisconsin shape:", X_w.shape)
print("Positive class rate:", round(y_w.mean(), 3))
```

```
Wisconsin shape: (569, 30)
Positive class rate: 0.373
```

[4]:
```
wisconsin_rows = []

wisconsin_linear_pipe = Pipeline(
    [
        ("scaler", StandardScaler()),
        ("model", SVC(kernel="linear", probability=True,␣
 ↪random_state=RANDOM_STATE)),
    ]
)
wisconsin_linear_search = GridSearchCV(
    wisconsin_linear_pipe,
    {"model__C": [0.1, 1, 10]},
    scoring="roc_auc",
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE),
    n_jobs=-1,
)
wisconsin_linear_search.fit(Xw_train, yw_train)

wisconsin_linear_pred = wisconsin_linear_search.best_estimator_.predict(Xw_test)
```

```
wisconsin_linear_score = wisconsin_linear_search.best_estimator_.
 ↪predict_proba(Xw_test)[:, 1]

wisconsin_rows.append(
    {
        "dataset": "Wisconsin",
        "kernel": "linear",
        "accuracy": accuracy_score(yw_test, wisconsin_linear_pred),
        "precision": precision_score(yw_test, wisconsin_linear_pred),
        "recall": recall_score(yw_test, wisconsin_linear_pred),
        "f1": f1_score(yw_test, wisconsin_linear_pred),
        "roc_auc": roc_auc_score(yw_test, wisconsin_linear_score),
        "pr_auc": average_precision_score(yw_test, wisconsin_linear_score),
        "cv_roc_auc": wisconsin_linear_search.best_score_,
        "predicted_positive_rate": wisconsin_linear_pred.mean(),
        "best_params": wisconsin_linear_search.best_params_,
    }
)

wisconsin_rbf_pipe = Pipeline(
    [
        ("scaler", StandardScaler()),
        ("model", SVC(kernel="rbf", probability=True,␣
 ↪random_state=RANDOM_STATE)),
    ]
)
wisconsin_rbf_search = GridSearchCV(
    wisconsin_rbf_pipe,
    {"model__C": [1, 10], "model__gamma": ["scale", 0.01]},
    scoring="roc_auc",
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE),
    n_jobs=-1,
)
wisconsin_rbf_search.fit(Xw_train, yw_train)

wisconsin_rbf_pred = wisconsin_rbf_search.best_estimator_.predict(Xw_test)
wisconsin_rbf_score = wisconsin_rbf_search.best_estimator_.
 ↪predict_proba(Xw_test)[:, 1]

wisconsin_rows.append(
    {
        "dataset": "Wisconsin",
        "kernel": "rbf",
        "accuracy": accuracy_score(yw_test, wisconsin_rbf_pred),
        "precision": precision_score(yw_test, wisconsin_rbf_pred),
        "recall": recall_score(yw_test, wisconsin_rbf_pred),
        "f1": f1_score(yw_test, wisconsin_rbf_pred),
```

```
        "roc_auc": roc_auc_score(yw_test, wisconsin_rbf_score),
        "pr_auc": average_precision_score(yw_test, wisconsin_rbf_score),
        "cv_roc_auc": wisconsin_rbf_search.best_score_,
        "predicted_positive_rate": wisconsin_rbf_pred.mean(),
        "best_params": wisconsin_rbf_search.best_params_,
    }
)

wisconsin_results = pd.DataFrame(wisconsin_rows)
wisconsin_results.round(4)
```

[4]:

	dataset	kernel	accuracy	precision	recall	f1	roc_auc	pr_auc	cv_roc_auc	predicted_positive_rate	best_params
0	Wisconsin	linear	0.9825	1.0	0.9524	0.9756	0.9964	0.9951	0.9944	0.3509	{'model__C': 0.1}
1	Wisconsin	rbf	0.9825	1.0	0.9524	0.9756	0.9960	0.9947	0.9954	0.3509	{'model__C': 10, 'model__gamma': 0.01}

Wisconsin ended up being almost too easy. After scaling, both kernels performed nearly

Brazil dataset

Brazil is much larger and much more imbalanced, so I used an 8,000-row sample to keep

[5]:
```
df_b, target_b = load_brazil(sample_size=8000, random_state=RANDOM_STATE)
X_b, y_b = split_X_y(df_b, target_b)

Xb_train, Xb_test, yb_train, yb_test = train_test_split(
    X_b,
    y_b,
    test_size=0.2,
    stratify=y_b,
    random_state=RANDOM_STATE,
)

print("Brazil sample shape:", X_b.shape)
print("Positive class rate:", round(y_b.mean(), 3))
```

```
Brazil sample shape: (8000, 39)
Positive class rate: 0.07
```

[6]:
```
brazil_rows = []

brazil_linear_pipe = Pipeline(
    [
        ("scaler", StandardScaler()),
```

```
        ("model", SVC(kernel="linear", probability=True,␣
 ↪random_state=RANDOM_STATE)),
    ]
)
brazil_linear_search = GridSearchCV(
    brazil_linear_pipe,
    {"model__C": [0.1, 1]},
    scoring="roc_auc",
    cv=StratifiedKFold(n_splits=3, shuffle=True, random_state=RANDOM_STATE),
    n_jobs=-1,
)
brazil_linear_search.fit(Xb_train, yb_train)

brazil_linear_pred = brazil_linear_search.best_estimator_.predict(Xb_test)
brazil_linear_score = brazil_linear_search.best_estimator_.
 ↪predict_proba(Xb_test)[:, 1]

brazil_rows.append(
    {
        "dataset": "Brazil",
        "kernel": "linear",
        "accuracy": accuracy_score(yb_test, brazil_linear_pred),
        "precision": precision_score(yb_test, brazil_linear_pred,␣
 ↪zero_division=0),
        "recall": recall_score(yb_test, brazil_linear_pred, zero_division=0),
        "f1": f1_score(yb_test, brazil_linear_pred, zero_division=0),
        "roc_auc": roc_auc_score(yb_test, brazil_linear_score),
        "pr_auc": average_precision_score(yb_test, brazil_linear_score),
        "cv_roc_auc": brazil_linear_search.best_score_,
        "predicted_positive_rate": brazil_linear_pred.mean(),
        "best_params": brazil_linear_search.best_params_,
    }
)

brazil_rbf_pipe = Pipeline(
    [
        ("scaler", StandardScaler()),
        ("model", SVC(kernel="rbf", probability=True,␣
 ↪random_state=RANDOM_STATE)),
    ]
)
brazil_rbf_search = GridSearchCV(
    brazil_rbf_pipe,
    {"model__C": [1], "model__gamma": ["scale", 0.01]},
    scoring="roc_auc",
    cv=StratifiedKFold(n_splits=3, shuffle=True, random_state=RANDOM_STATE),
    n_jobs=-1,
```

```
)
brazil_rbf_search.fit(Xb_train, yb_train)

brazil_rbf_pred = brazil_rbf_search.best_estimator_.predict(Xb_test)
brazil_rbf_score = brazil_rbf_search.best_estimator_.predict_proba(Xb_test)[:,␣
↪1]

brazil_rows.append(
    {
        "dataset": "Brazil",
        "kernel": "rbf",
        "accuracy": accuracy_score(yb_test, brazil_rbf_pred),
        "precision": precision_score(yb_test, brazil_rbf_pred, zero_division=0),
        "recall": recall_score(yb_test, brazil_rbf_pred, zero_division=0),
        "f1": f1_score(yb_test, brazil_rbf_pred, zero_division=0),
        "roc_auc": roc_auc_score(yb_test, brazil_rbf_score),
        "pr_auc": average_precision_score(yb_test, brazil_rbf_score),
        "cv_roc_auc": brazil_rbf_search.best_score_,
        "predicted_positive_rate": brazil_rbf_pred.mean(),
        "best_params": brazil_rbf_search.best_params_,
    }
)

brazil_results = pd.DataFrame(brazil_rows)
brazil_results.round(4)
```

[6]:

	dataset	kernel	accuracy	precision	recall	f1	roc_auc	pr_auc
0	Brazil	linear	0.9444	0.7143	0.3540	0.4734	0.8779	0.4584
1	Brazil	rbf	0.9462	0.7647	0.3451	0.4756	0.9330	0.5607

	cv_roc_auc	predicted_positive_rate	best_params
0	0.9100	0.0350	{'model__C': 1}
1	0.9249	0.0319	{'model__C': 1, 'model__gamma': 0.01}

Brazil was more informative. Accuracy stayed high for both kernels, but the recall numb

0.2 Combined comparison

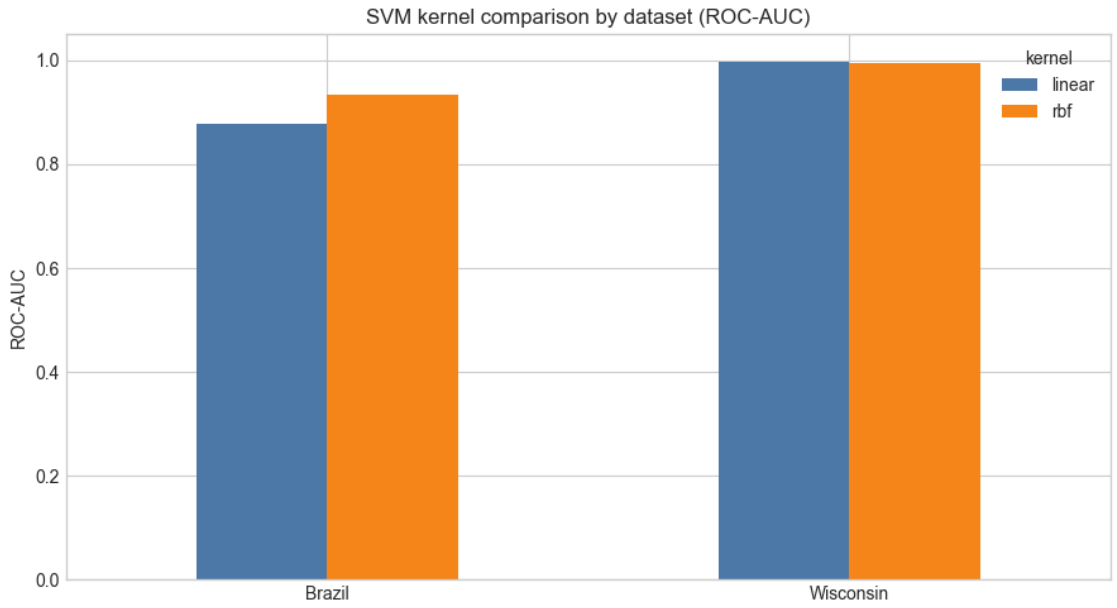
```
svm_results = pd.concat([wisconsin_results, brazil_results], ignore_index=True)
svm_results.round(4)
```

[7]:

	dataset	kernel	accuracy	precision	recall	f1	roc_auc	pr_auc
0	Wisconsin	linear	0.9825	1.0000	0.9524	0.9756	0.9964	0.9951
1	Wisconsin	rbf	0.9825	1.0000	0.9524	0.9756	0.9960	0.9947
2	Brazil	linear	0.9444	0.7143	0.3540	0.4734	0.8779	0.4584
3	Brazil	rbf	0.9462	0.7647	0.3451	0.4756	0.9330	0.5607

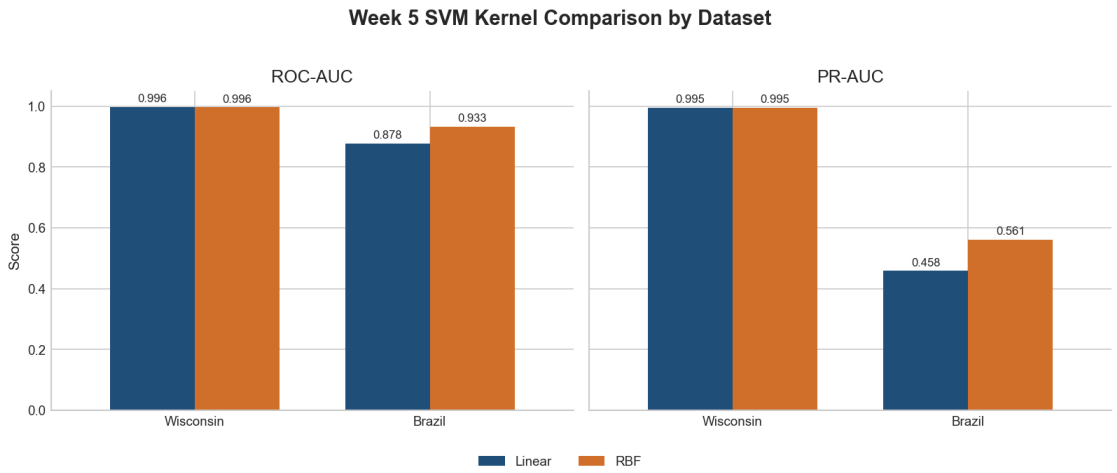
	cv_roc_auc	predicted_positive_rate	best_params
0	0.9944	0.3509	{'model__C': 0.1}
1	0.9954	0.3509	{'model__C': 10, 'model__gamma': 0.01}
2	0.9100	0.0350	{'model__C': 1}
3	0.9249	0.0319	{'model__C': 1, 'model__gamma': 0.01}

[8]:
```
roc_plot = svm_results.pivot(index="dataset", columns="kernel",␣
 ↪values="roc_auc")
ax = roc_plot.plot(kind="bar", figsize=(9, 5), color=["#4c78a8", "#f58518"],␣
 ↪rot=0)
ax.set_title("SVM kernel comparison by dataset (ROC-AUC)")
ax.set_ylabel("ROC-AUC")
ax.set_xlabel("")
ax.set_ylim(0, 1.05)
plt.tight_layout()
plt.show()
```



[9]:
```
import matplotlib.pyplot as plt

plt.style.use("seaborn-v0_8-whitegrid")
plt.rcParams.update({
    "figure.dpi": 150,
    "savefig.dpi": 300,
    "font.size": 10,
    "axes.titlesize": 12,
```

```
    "axes.labelsize": 10,
    "xtick.labelsize": 9,
    "ytick.labelsize": 9,
    "legend.fontsize": 9,
})
FIGURE_DIR = PROJECT_ROOT / "outputs" / "figures" / "milestone1"
FIGURE_DIR.mkdir(parents=True, exist_ok=True)

fig, axes = plt.subplots(1, 2, figsize=(10.8, 4.3), sharey=True)
metrics = [("roc_auc", "ROC-AUC"), ("pr_auc", "PR-AUC")]
colors = ["#1F4E79", "#D06F2A"]

for ax, (metric, label) in zip(axes, metrics):
    plot_df = svm_results.pivot(index="dataset", columns="kernel",␣
 ↪values=metric).loc[["Wisconsin", "Brazil"]]
    plot_df = plot_df[["linear", "rbf"]]
    plot_df.plot(kind="bar", ax=ax, color=colors, rot=0, width=0.72,␣
 ↪legend=False)
    ax.set_title(label)
    ax.set_xlabel("")
    ax.set_ylabel("Score")
    ax.set_ylim(0, 1.05)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    for container in ax.containers:
        ax.bar_label(container, fmt="%.3f", fontsize=8, padding=2)

handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, ["Linear", "RBF"], loc="lower center", ncol=2,␣
 ↪frameon=False, bbox_to_anchor=(0.5, -0.02))
fig.suptitle("Week 5 SVM Kernel Comparison by Dataset", fontsize=14,␣
 ↪weight="bold", y=1.02)
fig.tight_layout(rect=(0, 0.06, 1, 1))
fig.savefig(FIGURE_DIR / "fig05_svm_kernel_comparison.png",␣
 ↪bbox_inches="tight", facecolor="white")
plt.show()
```



Week 5 takeaway

For Wisconsin, SVM worked very well but the nonlinear kernel did not really buy much. I

For Brazil, the RBF kernel helped more, especially on ROC-AUC and PR-AUC, which suggest

06_week6_trees_random_forest

June 28, 2026

Each week, you will apply the concepts of that week to your Integrated Capstone Project

For Week 6, the focus is decision trees and random forests. This notebook compares both

[1]:
```
from pathlib import Path
import sys

PROJECT_ROOT = Path.cwd().resolve().parents[1]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.append(str(PROJECT_ROOT))

PROJECT_ROOT
```

[1]:
```
PosixPath('/Users/jamieconner/projects/bu/dx799_01/mod_c_capstone')
```

0.1 Imports

[2]:
```
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    average_precision_score,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
)
from sklearn.model_selection import GridSearchCV, StratifiedKFold,␣
 ↪train_test_split
from sklearn.tree import DecisionTreeClassifier

from src.load_data import load_brazil, load_wisconsin, split_X_y

plt.style.use("seaborn-v0_8-whitegrid")
pd.set_option("display.max_colwidth", 80)
RANDOM_STATE = 42
```

Wisconsin dataset

Wisconsin is a good first check because the classes are fairly clean. I wanted to see w

[3]:
```
df_w, target_w = load_wisconsin()
X_w, y_w = split_X_y(df_w, target_w)

Xw_train, Xw_test, yw_train, yw_test = train_test_split(
    X_w,
    y_w,
    test_size=0.2,
    stratify=y_w,
    random_state=RANDOM_STATE,
)

print("Wisconsin shape:", X_w.shape)
print("Positive class rate:", round(y_w.mean(), 3))
```

```
Wisconsin shape: (569, 30)
Positive class rate: 0.373
```

[4]:
```
wisconsin_rows = []

wisconsin_tree_search = GridSearchCV(
    DecisionTreeClassifier(random_state=RANDOM_STATE),
    {
        "max_depth": [3, 5, None],
        "min_samples_leaf": [1, 5, 10],
    },
    scoring="roc_auc",
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE),
    n_jobs=-1,
)
wisconsin_tree_search.fit(Xw_train, yw_train)

wisconsin_tree_model = wisconsin_tree_search.best_estimator_
wisconsin_tree_pred = wisconsin_tree_model.predict(Xw_test)
wisconsin_tree_prob = wisconsin_tree_model.predict_proba(Xw_test)[:, 1]

wisconsin_rows.append(
    {
        "dataset": "Wisconsin",
        "model": "decision_tree",
        "accuracy": accuracy_score(yw_test, wisconsin_tree_pred),
        "precision": precision_score(yw_test, wisconsin_tree_pred),
        "recall": recall_score(yw_test, wisconsin_tree_pred),
        "f1": f1_score(yw_test, wisconsin_tree_pred),
```

```
        "roc_auc": roc_auc_score(yw_test, wisconsin_tree_prob),
        "pr_auc": average_precision_score(yw_test, wisconsin_tree_prob),
        "train_roc_auc": roc_auc_score(
            yw_train,
            wisconsin_tree_model.predict_proba(Xw_train)[:, 1],
        ),
        "cv_roc_auc": wisconsin_tree_search.best_score_,
        "best_params": wisconsin_tree_search.best_params_,
    }
)

wisconsin_forest_search = GridSearchCV(
    RandomForestClassifier(random_state=RANDOM_STATE, n_jobs=-1),
    {
        "n_estimators": [100, 200],
        "max_depth": [5, None],
        "min_samples_leaf": [1, 5],
    },
    scoring="roc_auc",
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE),
    n_jobs=-1,
)
wisconsin_forest_search.fit(Xw_train, yw_train)

wisconsin_forest_model = wisconsin_forest_search.best_estimator_
wisconsin_forest_pred = wisconsin_forest_model.predict(Xw_test)
wisconsin_forest_prob = wisconsin_forest_model.predict_proba(Xw_test)[:, 1]

wisconsin_rows.append(
    {
        "dataset": "Wisconsin",
        "model": "random_forest",
        "accuracy": accuracy_score(yw_test, wisconsin_forest_pred),
        "precision": precision_score(yw_test, wisconsin_forest_pred),
        "recall": recall_score(yw_test, wisconsin_forest_pred),
        "f1": f1_score(yw_test, wisconsin_forest_pred),
        "roc_auc": roc_auc_score(yw_test, wisconsin_forest_prob),
        "pr_auc": average_precision_score(yw_test, wisconsin_forest_prob),
        "train_roc_auc": roc_auc_score(
            yw_train,
            wisconsin_forest_model.predict_proba(Xw_train)[:, 1],
        ),
        "cv_roc_auc": wisconsin_forest_search.best_score_,
        "best_params": wisconsin_forest_search.best_params_,
    }
)
```

```
wisconsin_results = pd.DataFrame(wisconsin_rows)
wisconsin_results.round(4)
```

[4]:

	dataset	model	accuracy	precision	recall	f1	roc_auc	pr_auc	train_roc_auc	cv_roc_auc	best_params
0	Wisconsin	decision_tree	0.9123	1.0	0.7619	0.8649	0.9688	0.9533	0.9929	0.9559	{'max_depth': 5, 'min_samples_leaf': 10}
1	Wisconsin	random_forest	0.9737	1.0	0.9286	0.9630	0.9929	0.9893	1.0000	0.9889	{'max_depth': None, 'min_samples_leaf': 1, 'n_estimators': 100}

Wisconsin still favored the random forest. The single tree worked reasonably well, but

Brazil dataset

Brazil is tougher and more imbalanced, so I used the same 8,000-row sample as Week 5 to

[5]:
```
df_b, target_b = load_brazil(sample_size=8000, random_state=RANDOM_STATE)
X_b, y_b = split_X_y(df_b, target_b)

Xb_train, Xb_test, yb_train, yb_test = train_test_split(
    X_b,
    y_b,
    test_size=0.2,
    stratify=y_b,
    random_state=RANDOM_STATE,
)

print("Brazil sample shape:", X_b.shape)
print("Positive class rate:", round(y_b.mean(), 3))
```

```
Brazil sample shape: (8000, 39)
Positive class rate: 0.07
```

[6]:
```
brazil_rows = []

brazil_tree_search = GridSearchCV(
    DecisionTreeClassifier(random_state=RANDOM_STATE),
    {
        "max_depth": [3, 5, None],
        "min_samples_leaf": [1, 5, 10],
    },
    scoring="roc_auc",
```

```
    cv=StratifiedKFold(n_splits=3, shuffle=True, random_state=RANDOM_STATE),
    n_jobs=-1,
)
brazil_tree_search.fit(Xb_train, yb_train)

brazil_tree_model = brazil_tree_search.best_estimator_
brazil_tree_pred = brazil_tree_model.predict(Xb_test)
brazil_tree_prob = brazil_tree_model.predict_proba(Xb_test)[:, 1]

brazil_rows.append(
    {
        "dataset": "Brazil",
        "model": "decision_tree",
        "accuracy": accuracy_score(yb_test, brazil_tree_pred),
        "precision": precision_score(yb_test, brazil_tree_pred,␣
 ↪zero_division=0),
        "recall": recall_score(yb_test, brazil_tree_pred, zero_division=0),
        "f1": f1_score(yb_test, brazil_tree_pred, zero_division=0),
        "roc_auc": roc_auc_score(yb_test, brazil_tree_prob),
        "pr_auc": average_precision_score(yb_test, brazil_tree_prob),
        "train_roc_auc": roc_auc_score(
            yb_train,
            brazil_tree_model.predict_proba(Xb_train)[:, 1],
        ),
        "cv_roc_auc": brazil_tree_search.best_score_,
        "best_params": brazil_tree_search.best_params_,
    }
)

brazil_forest_search = GridSearchCV(
    RandomForestClassifier(random_state=RANDOM_STATE, n_jobs=-1),
    {
        "n_estimators": [100, 200],
        "max_depth": [5, None],
        "min_samples_leaf": [1, 5],
    },
    scoring="roc_auc",
    cv=StratifiedKFold(n_splits=3, shuffle=True, random_state=RANDOM_STATE),
    n_jobs=-1,
)
brazil_forest_search.fit(Xb_train, yb_train)

brazil_forest_model = brazil_forest_search.best_estimator_
brazil_forest_pred = brazil_forest_model.predict(Xb_test)
brazil_forest_prob = brazil_forest_model.predict_proba(Xb_test)[:, 1]

brazil_rows.append(
```

```
    {
        "dataset": "Brazil",
        "model": "random_forest",
        "accuracy": accuracy_score(yb_test, brazil_forest_pred),
        "precision": precision_score(yb_test, brazil_forest_pred,␣
 ↪zero_division=0),
        "recall": recall_score(yb_test, brazil_forest_pred, zero_division=0),
        "f1": f1_score(yb_test, brazil_forest_pred, zero_division=0),
        "roc_auc": roc_auc_score(yb_test, brazil_forest_prob),
        "pr_auc": average_precision_score(yb_test, brazil_forest_prob),
        "train_roc_auc": roc_auc_score(
            yb_train,
            brazil_forest_model.predict_proba(Xb_train)[:, 1],
        ),
        "cv_roc_auc": brazil_forest_search.best_score_,
        "best_params": brazil_forest_search.best_params_,
    }
)

brazil_results = pd.DataFrame(brazil_rows)
brazil_results.round(4)
```

[6]:

	dataset	model	accuracy	precision	recall	f1	roc_auc	pr_auc	train_roc_auc	cv_roc_auc	best_params
0	Brazil	decision_tree	0.9356	0.6042	0.2566	0.3602	0.8840	0.4648	0.9129	0.8856	{'max_depth': 5, 'min_samples_leaf': 10}
1	Brazil	random_forest	0.9406	0.6875	0.2920	0.4099	0.9363	0.5429	0.9733	0.9339	{'max_depth': None, 'min_samples_leaf': 5, 'n_estimators': 200}

Brazil made the tree-vs-forest difference easier to see. Accuracy stayed fairly high fo

0.2 Combined comparison

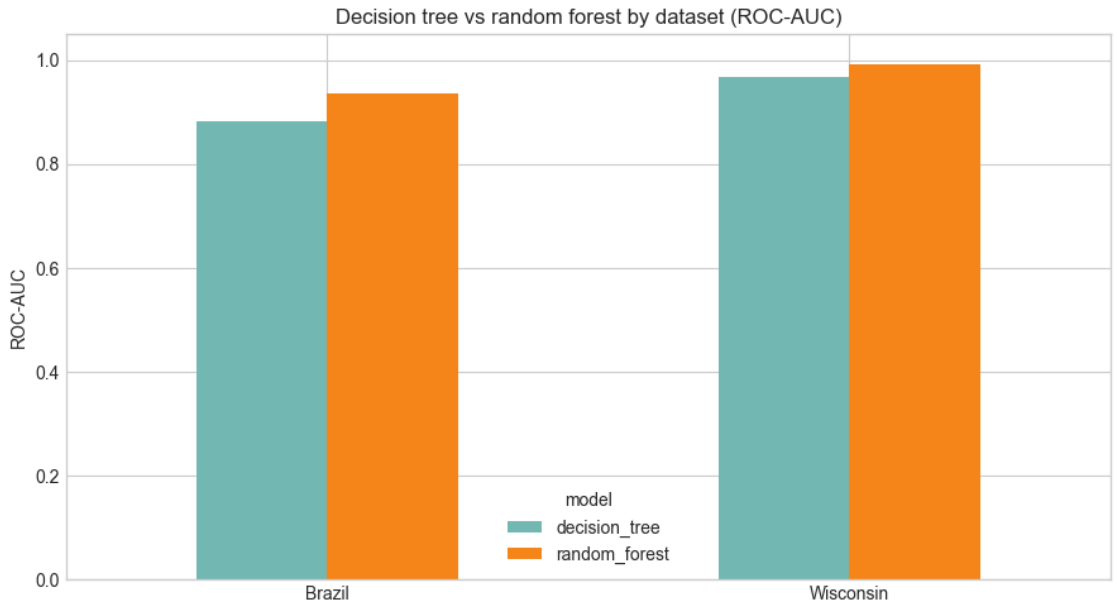
[7]:
```
week6_results = pd.concat([wisconsin_results, brazil_results],␣
 ↪ignore_index=True)
week6_results.round(4)
```

[7]:

	dataset	model	accuracy	precision	recall	f1	roc_auc	\
0	Wisconsin	decision_tree	0.9123	1.0000	0.7619	0.8649	0.9688	
1	Wisconsin	random_forest	0.9737	1.0000	0.9286	0.9630	0.9929	
2	Brazil	decision_tree	0.9356	0.6042	0.2566	0.3602	0.8840	

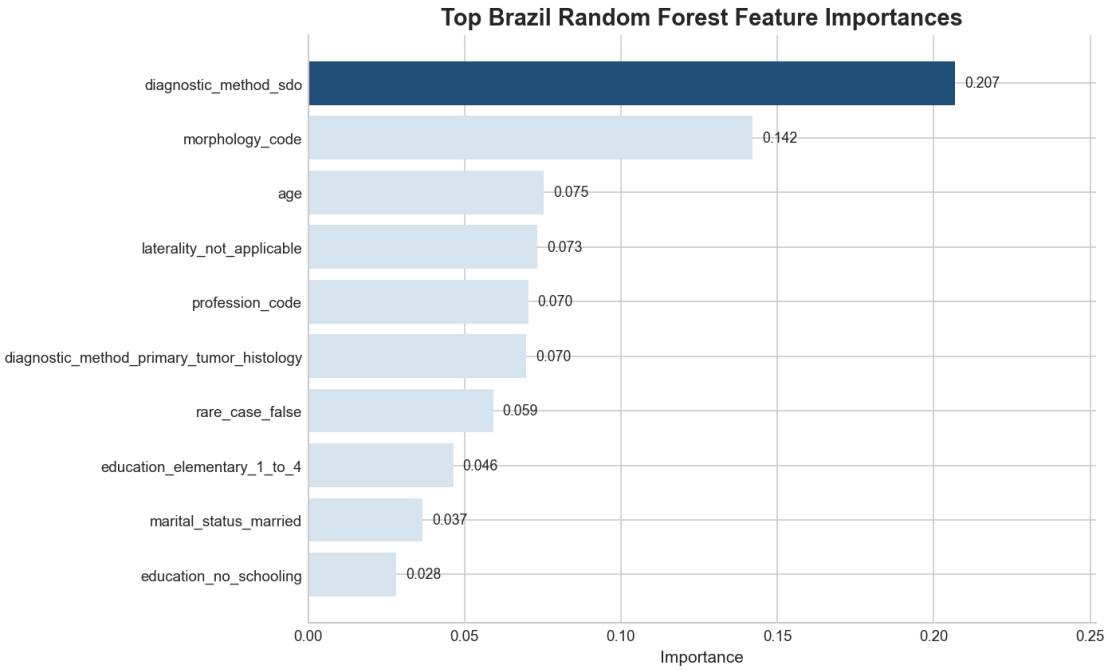
```
3     Brazil  random_forest    0.9406     0.6875  0.2920  0.4099   0.9363

   pr_auc  train_roc_auc  cv_roc_auc  \
0  0.9533         0.9929      0.9559
1  0.9893         1.0000      0.9889
2  0.4648         0.9129      0.8856
3  0.5429         0.9733      0.9339

                                                  best_params
0                       {'max_depth': 5, 'min_samples_leaf': 10}
1  {'max_depth': None, 'min_samples_leaf': 1, 'n_estimators': 100}
2                       {'max_depth': 5, 'min_samples_leaf': 10}
3  {'max_depth': None, 'min_samples_leaf': 5, 'n_estimators': 200}
```

[8]:
```
roc_plot = week6_results.pivot(index="dataset", columns="model",␣
 ↪values="roc_auc")
ax = roc_plot.plot(kind="bar", figsize=(9, 5), color=["#72b7b2", "#f58518"],␣
 ↪rot=0)
ax.set_title("Decision tree vs random forest by dataset (ROC-AUC)")
ax.set_ylabel("ROC-AUC")
ax.set_xlabel("")
ax.set_ylim(0, 1.05)
plt.tight_layout()
plt.show()
```



0.3 Brazil random forest feature importance

[9]:
```
import matplotlib.pyplot as plt

plt.style.use("seaborn-v0_8-whitegrid")
plt.rcParams.update({
    "figure.dpi": 150,
    "savefig.dpi": 300,
    "font.size": 10,
    "axes.titlesize": 12,
    "axes.labelsize": 10,
    "xtick.labelsize": 9,
    "ytick.labelsize": 9,
    "legend.fontsize": 9,
})
FIGURE_DIR = PROJECT_ROOT / "outputs" / "figures" / "milestone1"
FIGURE_DIR.mkdir(parents=True, exist_ok=True)

brazil_feature_importance = (
    pd.Series(brazil_forest_model.feature_importances_, index=X_b.columns)
    .sort_values(ascending=False)
    .head(10)
    .sort_values()
)

fig, ax = plt.subplots(figsize=(9.5, 5.8))
colors = ["#D6E4F0"] * len(brazil_feature_importance)
colors[-1] = "#1F4E79"
ax.barh(brazil_feature_importance.index, brazil_feature_importance.values,␣
 ↪color=colors)
ax.set_title("Top Brazil Random Forest Feature Importances", fontsize=14,␣
 ↪weight="bold")
ax.set_xlabel("Importance")
ax.set_ylabel("")
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
for y_pos, value in enumerate(brazil_feature_importance.values):
    ax.text(value + 0.003, y_pos, f"{value:.3f}", va="center", fontsize=8.5)
ax.set_xlim(0, brazil_feature_importance.max() + 0.045)

fig.tight_layout()
fig.savefig(FIGURE_DIR / "fig06_brazil_rf_feature_importance.png",␣
 ↪bbox_inches="tight", facecolor="white")
plt.show()

brazil_feature_importance.sort_values(ascending=False).round(4)
```



[9]:
```
diagnostic_method_sdo                          0.2069
morphology_code                                0.1421
age                                            0.0753
laterality_not_applicable                      0.0733
profession_code                                0.0704
diagnostic_method_primary_tumor_histology      0.0698
rare_case_false                                0.0591
education_elementary_1_to_4                    0.0464
marital_status_married                         0.0366
education_no_schooling                         0.0281
dtype: float64
```

The importance plot suggests that the forest relied most on a mix of diagnostic, morpho

Week 6 takeaway

For both datasets, the random forest outperformed the single decision tree. On Wisconsi

On Brazil, the difference mattered more. The tuned decision tree was limited enough to